

Zhen Qin¹, Zhaomin Wu^{2*}, Bingsheng He², Shuiguang Deng^{1*}秦震¹, 吴兆敏^{2*}, 何丙胜², 邓水光^{1*}¹ Zhejiang University, ² National University of Singapore
zhenqin@zju.edu.cn, zhaomin@nus.edu.sg,
hebs@comp.nus.edu.sg, dengsg@zju.edu.cn¹ 浙江大学, ² 新加坡国立大学 zhenqin@zju.edu.cn,
zhaomin@nus.edu.sg, hebs@comp.nus.edu.sg,
dengsg@zju.edu.cn

Abstract 摘要

Instruction tuning is a crucial step in improving the responsiveness of pretrained large language models (LLMs) to human instructions. Federated learning (FL) helps to exploit the use of vast private instruction data from clients, becoming popular for LLM tuning by improving data diversity. Existing federated tuning simply consumes all local data, causing excessive computational overhead and overfitting to local data, while centralized data-efficient solutions are not suitable for FL due to privacy concerns. This work presents FedHDS, a federated data-efficient instruction tuning approach, which tunes LLMs with a representative subset of edge-side data. It reduces the data redundancy at both intra- and inter-client levels without sharing raw data. Experiments with various LLMs, datasets and partitions show that FedHDS improves Rouge-L on unseen tasks by an average of 10.72% over the SOTA full-data federated instruction tuning methods, while using less than 1.5% of the data samples, improving training efficiency by up to tens of times.

指令微调是提高预训练大语言模型 (LLMs) 对人类指令响应能力的关键步骤。联邦学习 (FL) 有助于利用来自客户端的海量私有指令数据, 通过提高数据多样性, 在 LLM 微调中日益受到关注。现有的联邦微调方法只是简单地消耗所有本地数据, 这会导致过度的计算开销以及对本地数据的过拟合, 而中心化的数据高效方案由于隐私问题并不适用于 FL。本研究提出了 FedHDS, 一种联邦数据高效指令微调方法, 它利用边缘侧数据的代表性子集来微调 LLMs。该方法在不共享原始数据的情况下, 减少了客户端内部和客户端之间的数据冗余。在多种 LLMs、数据集和划分方式上的实验表明, FedHDS 在未知任务上的 Rouge-L 指标比最先进的全数据联邦指令微调方法平均提高了 10.72%, 同时仅使用了不到 1.5% 的数据样本, 将训练效率提升了高达数十倍。

1 Introduction 1 引言

Large language models (LLMs) exhibit remarkable performance on a wide range of natural language tasks. Instruction tuning (Wang et al., 2022) is crucial to improve LLMs' responsiveness to human instructions, whose success hinges on the availability of diverse and high-quality data (Wang et al., 2024, 2023; Li et al., 2024a), particularly for unseen tasks (Wei et al., 2022). As the pool of publicly available data is expected to be exhausted in the near future (Villalobos et al., 2022), exploiting more data sources, such as data on edge devices, becomes essential for continued LLM improvement (Qin et al., 2024a). However, privacy concerns and regulations such as GDPR (Voigt and Von dem Bussche, 2017) complicate the use of edge-side data.

大型语言模型 (LLMs) 在广泛的自然语言任务中表现出卓越的性能。指令微调 (Wang et al., 2022) 对于提高 LLMs 对人类指令的响应能力至关重要, 其成功取决于多样化且高质量数据的可用性 (Wang et al., 2024, 2023; Li et al., 2024a), 特别是对于未见过的任务 (Wei et al., 2022)。由于预计公众可用的数据池将在不久的将来耗尽 (Villalobos et al., 2022), 开发更多数据源 (如边缘设备上的数据) 对于 LLM 的持续改进变得至关重要 (Qin et al., 2024a)。然而, 隐私担忧和诸如 GDPR (Voigt and Von dem Bussche, 2017) 之类的法规使得边缘侧数据的使用变得复杂。

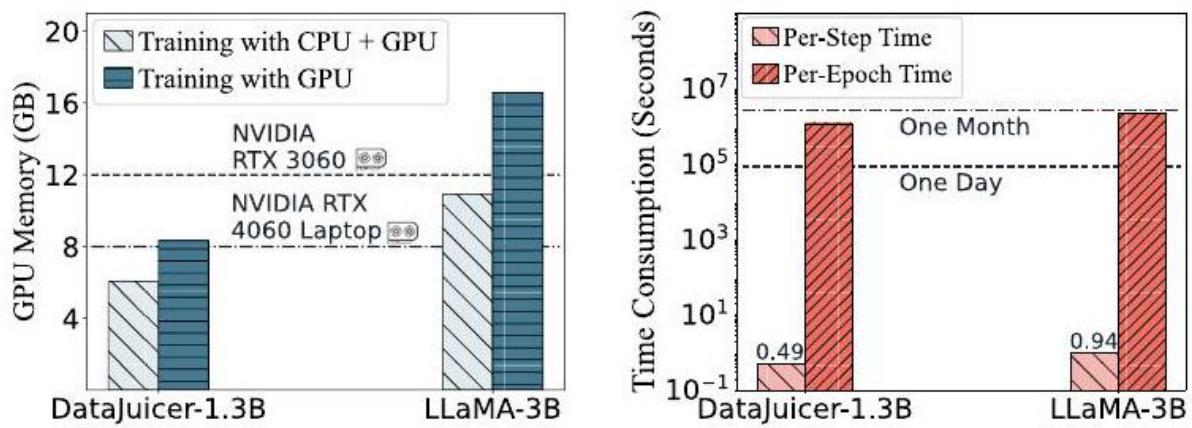


Figure 1: GPU mem- Figure 2: Per-step/epoch ory cost by training with time of BP with LoRA LoRA ¹ v.s. capacity of on Natural Instructions popular edge-side GPUs ². dataset by CPU+GPU.

图 1: 使用 LoRA 训练的 GPU 显存开销 ¹ 与常用边缘侧 GPU 容量的对比 ²。图 2: 在 Natural Instructions 数据集上, 通过 CPU+GPU 使用 LoRA 进行反向传播 (BP) 的每步/每轮耗时。

Federated learning (FL) (McMahan et al., 2017) has emerged as a promising solution to utilize diverse data from edge devices to tune LLMs (Zhang et al., 2024), especially for generalization to unseen tasks (Qin et al., 2024a; Bai et al., 2024). Prior FL works on LLMs mainly focus on communication and memory costs (Zhang et al., 2024; Babakniya et al., 2023; Zhang et al., 2023; Che et al., 2023; Kuang et al., 2024; Xu et al., 2024; Qin et al., 2024a). Although these methods show promising results, our investigation reveals that they often use all local data for training (Qin et al., 2024a; Xu et al., 2024; Zhang et al., 2024; Ling et al., 2024). This strategy leads to many training steps, and thus causes two main issues in FL contexts:

联邦学习 (FL) (McMahan et al., 2017) 已成为一种极具前景的解决方案, 可利用来自边缘设备的多元化数据来微调 LLMs (Zhang et al., 2024), 尤其是在提升对未知任务的泛化能力方面 (Qin et al., 2024a; Bai et al., 2024)。以往针对 LLMs 的联邦学习研究主要集中在通信和内存成本上 (Zhang et al., 2024; Babakniya et al., 2023; Zhang et al., 2023; Che et al., 2023; Kuang et al., 2024; Xu et al., 2024; Qin et al., 2024a)。尽管这些方法取得了不错的效果, 但我们的调查发现, 它们通常使用所有本地数据进行训练 (Qin et al., 2024a; Xu et al., 2024; Zhang et al., 2024; Ling et al., 2024)。这种策略会导致大量的训练步骤, 从而在联邦学习场景下引发两个主要问题:

(1) Efficiency: It is time-consuming to train LLMs, especially when edge devices have limited GPU capacity and thus require CPU+GPU hybrid computing (Figure 1), where iterating through the entire dataset may incur intolerable time consumption (Figure 2). (2) Generalization: FL clients usually hold data covering limited domains. Training on redundant data increases the risk of overfitting, harming generalization to unseen tasks. Thus, in FL, it is necessary to explore tuning LLMs using

(1) 效率: 训练 LLMs 非常耗时, 特别是当边缘设备 GPU 容量有限, 从而需要 CPU+GPU 混合计算时 (图 1), 遍历整个数据集可能会产生难以忍受的时间消耗 (图 2)。(2) 泛化性: 联邦学习 (FL) 客户端通常持有覆盖有限领域的的数据。在冗余数据上进行训练会增加过拟合的风险, 从而损害对未见过任务的泛化能力。因此, 在联邦学习中, 有必要探索使用

fewer representative data to improve efficiency and reduce overfitting, i.e., data-efficient instruction tuning (Chen et al., 2023; Sachdeva et al., 2024), which has been touched in centralized scenarios, e.g., tuning LLMs with only high-quality samples (Zhou et al., 2023; Li et al., 2024b; Lu et al., 2023; Cui et al., 2023) or representative samples (a.k.a. coreset) (Chen et al., 2023; Wu et al., 2023; Cao et al., 2024). Among them, coreset-based methods usually excel in reducing data volume.

更少的代表性数据来微调 LLMs, 以提高效率并减少过拟合, 即数据高效的指令微调 (Chen et al., 2023; Sachdeva et al., 2024)。这在中心化场景中已有涉及, 例如仅使用高质量样本 (Zhou et al., 2023; Li et al., 2024b; Lu et al., 2023; Cui et al., 2023) 或代表性样本 (又称核心集/coreset) (Chen et al., 2023; Wu et al., 2023; Cao et al., 2024) 来微调 LLMs。其中, 基于核心集的方法通常在减少数据量方面表现出色。

Despite their success in centralized scenarios, applying these works to FL faces two main limitations. (L1) Low compatibility with FL: Some methods access all data simultaneously (Chen et al., 2023; Wu et al., 2023; Cao et al., 2024), violating FL's fundamental principle of restricting direct access to data by third parties, and thus failing to detect inter-client data redundancy. (L2) Suboptimal data representations: The selection of coresets significantly impacts the tuning accuracy (Zha et al., 2024; Wang et al., 2024). Current methods rely on data representations from the last Transformer layer (Chen et al.,

2023; Wu et al., 2023; Cao et al., 2024), which may not capture the full spectrum of features to distinguish data samples.

尽管这些研究在中心化场景中取得了成功，但将其应用于联邦学习（FL）面临两个主要局限性。（L1）与 FL 的兼容性低：某些方法需要同时访问所有数据（Chen et al., 2023; Wu et al., 2023; Cao et al., 2024），这违反了 FL 限制第三方直接访问数据的基本原则，因此无法检测客户端之间的数据冗余。（L2）数据表示次优：核心集（coresets）的选择会显著影响微调精度（Zha et al., 2024; Wang et al., 2024）。现有方法依赖于 Transformer 最后一层的数据表示（Chen et al., 2023; Wu et al., 2023; Cao et al., 2024），这可能无法捕捉到区分数据样本的全方位特征。

Due to the limited research on federated dataefficient instruction tuning and the shortcomings of centralized methods in FL scenarios, we propose FedHDS, a novel framework of federated hierarchical data selection. It clusters local data on each client to detect intra-client data redundancy, and then sends approximate cluster centroids to the server for further clustering to identify inter-client data redundancy, cutting down computational costs and enhancing generalization to new tasks by taking the distribution of edge-side data without direct data access, which is compatible with FL (L1). Then, we propose to fuse data features from different Transformer layers, in order to provide better data representations for coreset selection (L2).

由于对联邦数据高效指令微调的研究有限，且中心化方法在 FL 场景中存在缺陷，我们提出了 FedHDS，这是一个全新的联邦分层数据选择框架。它在每个客户端上对本地数据进行聚类以检测客户端内部的数据冗余，然后将近似聚类质心发送到服务器进行进一步聚类，以识别客户端之间的数据冗余。该框架在不直接访问数据的情况下，通过考虑边缘侧数据的分布，降低了计算成本并增强了对新任务的泛化能力，从而与 FL 兼容 (L1)。随后，我们提出融合来自不同 Transformer 层的数据特征，以便为核心集选择提供更好的数据表示 (L2)。

This work makes the following contributions:

本研究的主要贡献如下：

We propose FedHDS, a framework that tunes LLMs using coreset while alleviating intra-client and inter-client data redundancy. To the best of our knowledge, this is the first study on federated data-efficient instruction tuning for LLMs.

我们提出了 FedHDS，这是一个利用核心集微调 LLMs 的框架，同时缓解了客户端内部和客户端之间的数据冗余。据我们所知，这是首个关于 LLMs 联邦数据高效指令微调的研究。

We propose a simple yet effective method to fuse features of varying abstraction levels from different Transformer layers. It works within FedHDS to facilitate clustering-based coreset selection for federated instruction tuning.

我们提出了一种简单而有效的方法，用于融合来自不同 Transformer 层、具有不同抽象层级的特征。该方法在 FedHDS 框架内运行，旨在促进基于聚类的核心集选择，以实现联邦指令微调。

We conduct experiments on two widely-used instruction datasets with various LLMs and nonIID partitions, showing that FedHDS improves Rouge-L on unseen tasks by 10.72% on average, using less than 1.5% of the data samples compared to existing practical federated baselines ¹.

我们在两个广泛使用的指令数据集上，针对多种 LLMs 和非独立同分布（non-IID）划分进行了实验。结果表明，与现有的实际联邦基准方法 ¹ 相比，FedHDS 在仅使用不到 1.5% 的数据样本的情况下，将未知任务上的 Rouge-L 平均提升了 10.72%。

2 Related Work 2 相关工作

Federated Learning for LLM Tuning Federated tuning for LLMs gains widespread attention by enabling edge-side data use without direct access. Due to the scale of LLMs, many studies develop federated tuning based on parameter-efficient fine-tuning (PEFT) techniques to reduce memory and communication costs (Zhang et al., 2024; Babakniya et al., 2023; Zhang et al., 2023; Che et al., 2023). Among PEFT techniques, LoRA (Hu et al., 2022a) gains significant attention, with studies on initialization (Babakniya et al., 2023), objective consistency (Sun et al., 2024), and heterogeneous client resources (Bai et al., 2024). Some works focus on specific costs, such as communication by transmitting only loss values and random seeds

(Qin et al., 2024a) and memory by employing quantization-aware training (Xu et al., 2024).

用于 LLM 微调的联邦学习：LLM 的联邦微调通过在不直接访问数据的情况下利用边缘侧数据，获得了广泛关注。由于 LLMs 的规模庞大，许多研究基于参数高效微调（PEFT）技术开发联邦微调，以降低内存和通信成本（Zhang et al., 2024; Babakniya et al., 2023; Zhang et al., 2023; Che et al., 2023）。在 PEFT 技术中，LoRA (Hu et al., 2022a) 备受瞩目，相关研究涉及初始化（Babakniya et al., 2023）、目标一致性（Sun et al., 2024）以及异构客户端资源（Bai et al., 2024）。一些工作专注于特定成本，例如通过仅传输损失值和随机种子来降低通信成本（Qin et al., 2024a），以及通过采用量化感知训练来降低内存成本（Xu et al., 2024）。

During local training, existing works typically consume all local data (Qin et al., 2024a; Xu et al., 2024; Zhang et al., 2024; Wu et al., 2024; Kuang et al., 2024; Sun et al., 2024), leading to a significant computational cost and overfitting to the local data, as client-side data in FL often consists of many samples from only a few domains. Unlike existing methods, this work takes a data-centric perspective (Zha et al., 2024), which focuses on data efficiency by selecting a small number of representative data samples for LLM tuning, to achieve better efficiency and generalization on unseen tasks.

在本地训练期间，现有工作通常消耗所有本地数据（Qin et al., 2024a; Xu et al., 2024; Zhang et al., 2024; Wu et al., 2024; Kuang et al., 2024; Sun et al., 2024），这导致了巨大的计算开销，并容易对本地数据产生过拟合，因为联邦学习中的客户端数据通常由来自少数领域的多个样本组成。与现有方法不同，本研究采用了以数据为中心（Zha et al., 2024）的视角，通过选择少量具有代表性的数据样本进行 LLM 微调，专注于数据效率，从而在未知任务上实现更好的效率和泛化能力。

Data-Efficient LLM Instruction Tuning Existing studies show that LLMs learn knowledge primarily from pretraining, and can be taught to produce better outputs with a limited amount of instruction data (Zhou et al., 2023). Some studies achieve comparable or better tuning results using fewer samples by focusing on data quality or the representativeness of data samples.

数据高效的 LLM 指令微调。现有研究表明，LLMs 主要从预训练中学习知识，并且可以通过有限数量的指令数据被教会产生更好的输出（Zhou et al., 2023）。一些研究通过关注数据质量或数据样本的代表性，使用更少的样本实现了相当甚至更好的微调效果。

Quality-oriented works either filter out lowquality samples by heuristics (Chen et al., 2024), quality indicators (Li et al., 2024c; Chen et al.,

以质量为导向的工作要么通过启发式方法（Chen et al., 2024）、质量指标（Li et al., 2024c; Chen et al.,

2024) or third-party LLMs (Li et al., 2024b), or curate high-quality samples by manual efforts (Zhou et al., 2023) or third-party LLMs (Lu et al., 2023; Cui et al., 2023). Methods based on heuristics or quality indicators can filter out a limited number of samples. Manual or third-party LLM-based methods are unsuitable for FL due to privacy risks.

2024) 或第三方 LLMs (Li et al., 2024b)，或者通过人工努力（Zhou et al., 2023）或第三方 LLMs (Lu et al., 2023; Cui et al., 2023) 筛选高质量样本。基于启发式方法或质量指标的方法只能过滤掉有限数量的样本。而基于人工或第三方 LLM 的方法由于存在隐私风险，并不适用于联邦学习（FL）。

Representativeness-based works select a few representative samples by clustering algorithms, filtering out much of the data while retaining tuning accuracy (e.g., more than 95% of the dataset) (Chen et al., 2023; Wu et al., 2023). However, existing coreset selection methods have shortcomings in FL contexts: 1) the nature of FL, where data stays on the device, prevents existing methods from being fully integrated, such as detecting data redundancy across clients. 2) existing methods often extract data features for clustering using only the last layer of the Transformer (Chen et al., 2023; Wu et al., 2023; Li et al., 2024c), which may result in a suboptimal feature space to distinguish coresets.

基于代表性的研究工作通过聚类算法选择少量具有代表性的样本，在过滤掉大部分数据的同时保留微调精度（例如，过滤掉超过 95% 的数据集）（Chen et al., 2023; Wu et al., 2023）。然而，现有的核心集选择方法在联邦学习（FL）场景中存在缺陷：1) 联邦学习数据保留在设备上的特性，阻碍了现有方法的全面整合，例如难以检测跨客户端的数据冗余。2) 现有方法通常仅使用 Transformer 的最后一层提取数据特征进行聚类（Chen et al., 2023; Wu et al., 2023; Li et al., 2024c），这可能导致用于区分核心集的特征空间并非最优。

Table 1: Table 1: Qualitative comparison between related instruction tuning methods for LLMs and FedHDS.

表 1：LLMs 相关指令微调方法与 FedHDS 的定性比较。

	Reduce Intra-Client Data Redundancy	Reduce Inter-Client Data Redundancy (Privacy-Preserving)	Full-Layer Feature Fusion
	减少客户端内数据冗余	减少客户端间数据冗余（隐私保护）	全层特征融合
Federated Instruction Tuning 联邦指令微调	$\times$	$\times$	$\times$
Centralized Data-Efficient Instruction Tuning 中心化数据高效指令微调	$\checkmark$	$\times$	$\times$
FedHDS (ours) FedHDS（本文方法）	$\checkmark$	$\checkmark$	$\checkmark$

Summary As Table 1, existing federated tuning overlooks data redundancy. Centralized dataefficient methods are either impractical for FL due to external data transmission or fail to address interclient redundancy (L1). They also rely on only the last-layer Transformer outputs to distinguish data samples, potentially causing sub-optimal data representations (L2). Unlike these methods, FedHDS performs instruction tuning using a coreset while alleviating both intra- and inter-client redundancy, with fused features from all Transformer layers.

总结如表 1 所示，现有的联邦微调忽略了数据冗余。中心化的数据高效方法要么因外部数据传输而不适用于联邦学习（FL），要么无法解决客户端间的冗余问题（L1）。此外，这些方法仅依赖 Transformer 最后一层的输出来区分数据样本，可能导致数据表示不够理想（L2）。与这些方法不同，FedHDS 使用核心集进行指令微调，同时缓解了客户端内部和客户端间的冗余，并融合了来自所有 Transformer 层的特征。

3 Problem Formulation 3 问题定义

Federated Instruction Tuning Assuming there are N clients in an FL system, each client i holds a private dataset $\mathcal{D}_i$ containing several instances of instruction data. Given an LLM $\mathbf{w} \in \mathbb{R}^d$ initialized by the pretrained weight $\mathbf{w}^0$, federated instruction tuning aims at optimizing $\mathbf{w}$ with the private data

联邦指令微调 假设联邦学习（FL）系统中有 N 个客户端，每个客户端 i 持有一个包含若干指令数据实例的私有数据集 $\mathcal{D}_i$ 。给定一个由预训练权重 $\mathbf{w}^0$ 初始化的 LLM $\mathbf{w} \in \mathbb{R}^d$ ，联邦指令微调旨在利用私有数据优化 $\mathbf{w}$

held by clients towards the following objective,

由客户端持有，目标如下：

$$\min_{\mathbf{w}} f(\mathbf{w}) \triangleq \sum_{i=1}^N \lambda_i \cdot \mathbb{E}_{\mathbf{x} \sim \mathcal{D}_i} [\mathcal{L}(\mathbf{w}; \mathbf{x})]$$

where $\mathcal{L}(\mathbf{w}; \mathbf{x})$ is the loss evaluated on model $\mathbf{w}$ for data instance $\mathbf{x}$ sampled from $\mathcal{D}_i$, and λ_i is the weight of client i that follows $\lambda_i > 0$ and $\sum_i \lambda_i = 1$. Symbol $\mathbf{x}$ is used as the batch size is 1 to reduce memory usage (Qin et al., 2024a). To solve Eq. (1), FL iterates multiple rounds. In each round r , several active clients get the latest model parameters $\mathbf{w}^r$ from the

server and perform several steps of stochastic gradient descent (SGD), as

其中 $\mathcal{L}(\mathbf{w}; \mathbf{x})$ 是在从 $\mathcal{D}_i$ 采样的数据实例 $\mathbf{x}$ 上对模型 $\mathbf{w}$ 评估的损失， λ_i 是遵循 $\lambda_i > 0$ 和 $\sum_i \lambda_i = 1$ 的客户端 i 的权重。由于批量大小为 1 以减少内存使用，因此使用了符号 $\mathbf{x}$ (Qin et al., 2024a)。为了求解等式 (1)，联邦学习 (FL) 会进行多轮迭代。在每一轮 r 中，若干活跃客户端从服务器获取最新的模型参数 $\mathbf{w}^r$ ，并执行若干步随机梯度下降 (SGD)，如下所示：

$$\mathbf{w}_{i,t+1}^r = \mathbf{w}_{i,t}^r - \eta \cdot \nabla_{\mathbf{w}_{i,t}^r} \mathcal{L}(\mathbf{w}_{i,t}^r; \mathbf{x}), \forall \mathbf{x} \in \mathcal{D}_i,$$

where $\mathbf{w}_{i,t}^r$ is the model of client i at the t -th local step in round r , and η is the learning rate. Typically, the process iterates over the local dataset $\mathcal{D}_i$ for one or more epochs (Qin et al., 2024a; Xu et al., 2024; Zhang et al., 2024). After local training, each active client sends the updated model to the server. To alleviate communication and memory costs, FL typically adopts PEFT techniques, where only a small subset of model parameters is trained and transmitted. Similarly to Zhang et al. (2024), this work only trains and transmits LoRA adapters.

其中 $\mathbf{w}_{i,t}^r$ 是客户端 i 在第 r 轮中第 t 个本地步骤的模型， η 是学习率。通常，该过程会在本地数据集 $\mathcal{D}_i$ 上迭代一个或多个轮次 (Epoch) (Qin et al., 2024a; Xu et al., 2024; Zhang et al., 2024)。本地训练结束后，每个活跃客户端将更新后的模型发送至服务器。为了减轻通信和内存开销，FL 通常采用参数高效微调 (PEFT) 技术，即仅训练和传输模型参数的一小分子集。与 Zhang et al. (2024) 类似，本研究仅训练并传输 LoRA 适配器。

Federated Data-Efficient Instruction Tuning

联邦数据高效指令微调

Assuming each client uses a data selection function $f(\mathbf{x}; \cdot) \rightarrow \{0, 1\}$ to construct a coreset

假设每个客户端使用数据选择函数 $f(\mathbf{x}; \cdot) \rightarrow \{0, 1\}$ 来构建核心集

$$\tilde{\mathcal{D}}_i = \{\mathbf{x} \in \mathcal{D}_i \mid f(\mathbf{x}; \cdot) = 1\},$$

and performs local training only on $\tilde{\mathcal{D}}_i$ as Eq. (2). If f makes the model $\tilde{\mathbf{w}}$ trained on $\{\tilde{\mathcal{D}}_i\}_{i=1}^N$ achieve accuracy comparable to-or even better than-that obtained with the full datasets, while satisfying

并仅在 $\tilde{\mathcal{D}}_i$ 上进行如等式 (2) 所示的本地训练。如果 f 使得在 $\{\tilde{\mathcal{D}}_i\}_{i=1}^N$ 上训练的模型 $\tilde{\mathbf{w}}$ 能够获得与全量数据集相当、甚至更好的准确率，同时满足

$$\sum_{i=1}^N |\tilde{\mathcal{D}}_i| / \sum_{i=1}^N |\mathcal{D}_i| \ll 1,$$

then the method is data-efficient, which greatly reduces the computation cost than the full-data ones.

那么该方法就是数据高效的，相比全量数据方法能大幅降低计算成本。

Additionally, by considering the redundancy among local coresets, we can further reduce the local iterations for each client. Assuming an ideal distribution $\mathcal{P}^*$ that each real-world data $\mathbf{x}$ follows (Qin et al., 2024b), and data samples in each coreset $\tilde{\mathcal{D}}_i$ follow $\mathcal{P}_i$, where local data distributions $\{\mathcal{P}_1, \dots, \mathcal{P}_N\}$ share mutual similarity to some extent since all the client-side

data can be regarded

此外，通过考虑本地核心集之间的冗余性，我们可以进一步减少每个客户端的本地迭代次数。假设存在一个理想分布 $\mathcal{P}^*$ ，每个真实世界数据 $\mathbf{x}$ 都遵循该分布 (Qin et al., 2024b)，且每个核心集 $\tilde{\mathcal{D}}_i$ 中的数据样本遵循 $\mathcal{P}_i$ ，其中本地数据分布 $\{\mathcal{P}_1, \dots, \mathcal{P}_N\}$ 在某种程度上具有相互相似性，因为所有客户端数据都可以被视为

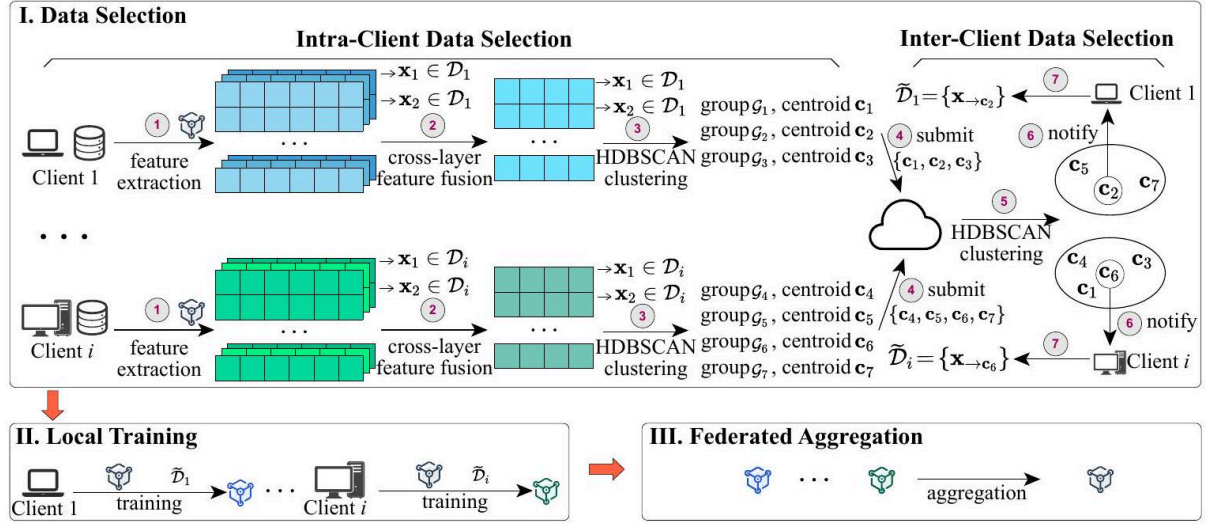


Figure 2: Figure 3: Overview of FedHDS in each round of FL. It differs from vanilla FL (McMahan et al., 2017) by incorporating "I. Data Selection" to select representative data samples before "II. Local Training". Centroid c is derived from HDBSCAN and does not correspond to specific samples. Data sample $\mathbf{x} \rightarrow c_j$ denotes the one closest to the centroid c_j of corresponding group $\mathcal{G}_j$. Set $\tilde{\mathcal{D}}_i$ contains representative samples in dataset $\mathcal{D}_i$ of the i -th client.

图 2: 图 3: FL 每轮中 FedHDS 的概述。它与原始 FL (McMahan et al., 2017) 的不同之处在于，在 "II. 本地训练" 之前加入了 "I. 数据选择" 以挑选具有代表性的数据样本。质心 c 由 HDBSCAN 导出，并不对应特定样本。数据样本 $\mathbf{x} \rightarrow c_j$ 表示最接近对应组 $\mathcal{G}_j$ 质心 c_j 的样本。集合 $\tilde{\mathcal{D}}_i$ 包含第 i 个客户端数据集 $\mathcal{D}_i$ 中的代表性样本。

as sampled from $\mathcal{P}^*$, so that inter-client data redundancy may exist. Experimental results in Section 5.4 also illustrate this. To alleviate intra-client and inter-client data redundancy, we design FedHDS.

由于是从 $\mathcal{P}^*$ 中采样的，因此可能存在客户端间的数据冗余。第 5.4 节的实验结果也说明了这一点。为了缓解客户端内和客户端间的数据冗余，我们设计了 FedHDS。

4 Approach 4 方法

4.1 Overview 4.1 概述

FedHDS identifies the representative data samples with their latent features. As shown in Figure 3, in round r , after downloading the global model $\mathbf{w}^r$, each client performs intra-client selection, followed by an inter-client selection performed by the server.

FedHDS 通过潜在特征识别具有代表性的数据样本。如图 3 所示，在第 r 轮中，在下载全局模型 $\mathbf{w}^r$ 后，每个客户端先进行客户端内选择，随后由服务器进行客户端间选择。

In intra-client selection, client i gets the hidden state in each Transformer layer with each data sample $\mathbf{x} \in \mathcal{D}_i$ as the input (1) in Figure 3). Then, these features are fused across different Transformer layers (2) in Figure 3). Next, clustering is performed to partition the data into several groups (3) in Figure 3). Each group holds an approximate centroid c that does not correspond to an individual data point. After that, the client sends these centroids $\{c_1, c_2, \dots\}$ to the server (4) in

Figure 3).

在客户端内选择中，客户端 i 以每个数据样本 $\mathbf{x} \in \mathcal{D}_i$ 作为输入，获取每个 Transformer 层的隐藏状态（图 3 中的 (1)）。然后，这些特征在不同的 Transformer 层之间进行融合（图 3 中的 (2)）。接着，通过聚类将数据划分为若干组（图 3 中的 (3)）。每个组持有一个近似质心 $\mathbf{c}$ ，该质心并不对应于单个数据点。之后，客户端将这些质心 $\{\mathbf{c}_1, \mathbf{c}_2, \dots, \}$ 发送到服务器（图 3 中的 (4)）。

Upon receiving the centroids from all active clients, inter-client selection starts. The server clusters received centroids into several groups (5) in Figure 3). In each group, the point closest to its centroid $\mathbf{c}_j^\Pi$ is designated as the chosen one. Then, the server notifies each client regarding which of their sent centroids are selected (6) in Figure 3). Next, each client i adds the data sample closest to each of the selected centroid within the corresponding

在接收到所有活动客户端的质心后，开始进行客户端间选择。服务器将接收到的质心聚类为若干组（图 3 中的 (5)）。在每个组中，最靠近其质心 $\mathbf{c}_j^\Pi$ 的点被指定为入选点。然后，服务器通知每个客户端其发送的哪些质心被选中（图 3 中的 (6)）。接着，每个客户端 i 将对应

group to coreset $\tilde{\mathcal{D}}_i$ (7) in Figure 3).

组内最接近每个被选质心的数据样本添加到核心集 $\tilde{\mathcal{D}}_i$ 中（图 3 中的 (7)）。

Finally, client i performs local training on $\tilde{\mathcal{D}}_i$ as Zhang et al. (2024). The selection processes are summarized in Algorithm 1 of Appendix A. In the following, we detail the design of FedHDS.

最后，客户端 i 按照 Zhang 等人 (2024) 的方法在 $\tilde{\mathcal{D}}_i$ 上进行本地训练。筛选过程总结在附录 A 的算法 1 中。在下文中，我们将详细介绍 FedHDS 的设计。

4.2 Intra-Client Data Selection

4.2 客户端内数据筛选

Data samples are selected based on features. Given an LLM with l Transformer layers, it can extract data features layer by layer and token by token, as

数据样本根据特征进行筛选。给定一个具有 l 个 Transformer 层的 LLM，它可以逐层、逐标记地提取数据特征，如下所示：

$$\begin{bmatrix} \mathbf{h}_j^{1,1} & \mathbf{h}_j^{1,2} & \dots & \mathbf{h}_j^{1,-1} \\ \mathbf{h}_j^{2,1} & \mathbf{h}_j^{2,2} & \dots & \mathbf{h}_j^{2,-1} \\ \dots & \dots & \dots & \dots \\ \mathbf{h}_j^{l,1} & \mathbf{h}_j^{l,2} & \dots & \mathbf{h}_j^{l,-1} \end{bmatrix},$$

where $\mathbf{h}_j^{l,b} \in \mathbb{R}^v$ is the hidden states of the b -th token from the l -th Transformer layer for the j th data sample, with $b = -1$ denoting the last token. From the token level, FedHDS uses the hidden state of the last token as Li et al. (2024c) since it encapsulates all preceding token information.

其中 $\mathbf{h}_j^{l,b} \in \mathbb{R}^v$ 是第 j 个数据样本中来自第 l 个 Transformer 层的第 b 个标记的隐藏状态， $b = -1$ 表示最后一个标记。在标记层面，FedHDS 采用 Li 等人 (2024c) 的做法，使用最后一个标记的隐藏状态，因为它封装了所有前序标记的信息。

At the layer level, prior works often use the last layer (Chen et al., 2023; Wu et al., 2023; Li et al., 2024c), which may not be universally optimal since different layers provide varying abstraction degrees to data representations. We show it with a toy example on Dolly-15 K, clustering its 8-category instructions into 8 groups using K-means. Figure 4 evaluates clustering

quality with Calinski-Harabasz Index (Caliński and Harabasz, 1974), and F_1 -score

在层级维度上，以往的研究通常使用最后一层（Chen et al., 2023; Wu et al., 2023; Li et al., 2024c），但这可能并非普遍最优，因为不同层对数据表示提供了不同程度的抽象。我们通过 Dolly-15K 的一个简单示例来展示这一点，使用 K-means 将其 8 类指令聚类为 8 组。图 4 使用 Calinski-Harabasz 指数 (Caliński and Harabasz, 1974) 和 F_1 分数评估了聚类质量

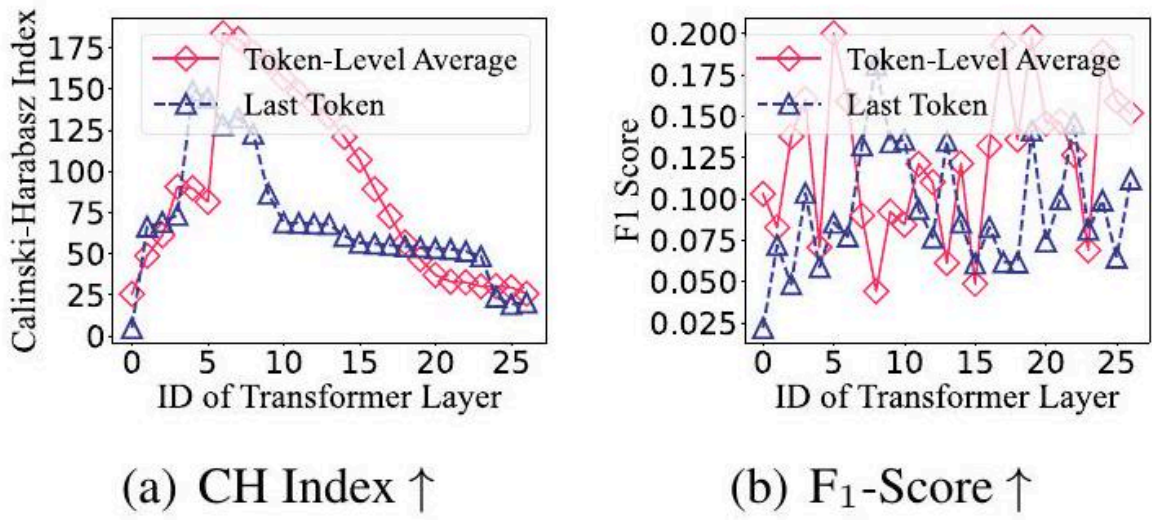


Figure 3: Figure 4: Evaluations of clustered data groups based on features from different Transformer layers, obtained in a centralized scenario with LLaMA-3B on Dolly-15K.

图 3: 图 4: 在中心化场景下，基于 LLaMA-3B 在 Dolly-15K 上提取的不同 Transformer 层特征对聚类数据组进行的评估。

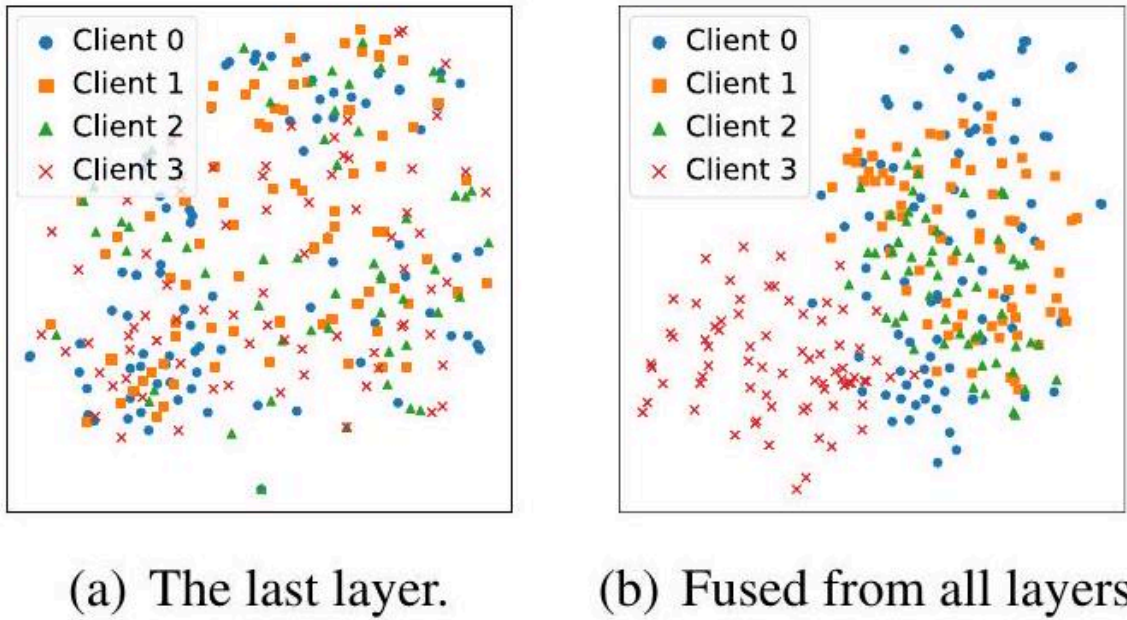


Figure 4: Figure 5: Visualization of features obtained with DataJuicer-1.3B on Dolly-15K ($\alpha = 5.0$).

图 4: 图 5: 使用 DataJuicer-1.3B 在 Dolly-15K ($\alpha = 5.0$) 上提取的特征可视化。

(more evaluations are left in Appendix E.1), showing that the last layer is not universally optimal, and no single layer excels across all metrics.

(更多评估见附录 E.1)，结果表明最后一层并非普遍最优，且没有哪一层能在所有指标上都表现出色。

Predicting the optimal layer is challenging, and computing clustering metrics for all layers is costly. A feasible approach is fusing features from all layers. One naive method is concatenating the last token's hidden states across layers, as

预测最优层具有挑战性，且计算所有层的聚类指标成本高昂。一种可行的方法是融合所有层的特征。一种朴素的方法是拼接各层最后一个 token 的隐藏状态，如下所示：

$$\mathbf{h}_j = [\mathbf{h}_j^{1,-1}, \mathbf{h}_j^{2,-1}, \dots, \mathbf{h}_j^{l,-1}].$$

However, Figure 4 shows that some layers degrade data separability. Dimensionality reduction may alleviate the impact of inappropriate dimensions, e.g., low-variance dimensions have minimal impact on distance calculation in t-SNE (Van der Maaten and Hinton, 2008). Thus, we apply $P: \mathbb{R}^{l \times v} \rightarrow \mathbb{R}^k, k \ll l \times v$ to fuse the features from l layers, as

然而，图 4 显示某些层会降低数据的可分性。降维可能会减轻不当维度的影响，例如，在 t-SNE (Van der Maaten 和 Hinton, 2008) 中，低方差维度对距离计算的影响微乎其微。因此，我们应用 $P: \mathbb{R}^{l \times v} \rightarrow \mathbb{R}^k, k \ll l \times v$ 来融合来自 l 层的特征，如下所示：

$$\{\tilde{\mathbf{h}}_1, \tilde{\mathbf{h}}_2, \dots, \tilde{\mathbf{h}}_{|\mathcal{D}_i|}\} = P(\{\mathbf{h}_1, \mathbf{h}_2, \dots, \mathbf{h}_{|\mathcal{D}_i|}\}).$$

We choose t-SNE with Barnes-Hut implementation (van der Maaten, 2013), as it is more effective in nonlinear spaces than earlier methods such as PCA (Jolliffe, 2002). The fused feature dimension k is set to 2 for efficiency. Figure 5 shows the effectiveness of feature fusion, where a random subset of Dolly-15 K is partitioned to 4 clients with label distribution skew. If last-layer features are employed,

我们选择采用 Barnes-Hut 实现的 t-SNE (van der Maaten, 2013)，因为它在非线性空间中比 PCA (Jolliffe, 2002) 等早期方法更有效。为了提高效率，融合后的特征维度 k 设置为 2。图 5 展示了特征融合的有效性，其中 Dolly-15K 的随机子集被分配给 4 个具有标签分布偏移的客户端。如果采用最后一层特征，

data among clients tends to be scattered, while fused features form relatively clear boundaries between clients, enhancing sample distinction.

客户端之间的数据往往比较分散，而融合特征在客户端之间形成了相对清晰的边界，增强了样本的区分度。

After obtaining fused features of local data $\mathcal{D}_i$, denoted by $\{\tilde{\mathbf{h}}_1, \tilde{\mathbf{h}}_2, \dots, \tilde{\mathbf{h}}_{|\mathcal{D}_i|}\}$, HDBSCAN (Campello et al., 2013) is applied to cluster them into groups $\{\mathcal{G}_1, \mathcal{G}_2, \dots\}$. Each group $\mathcal{G}_j$ holds a centroid $\mathbf{c}_j$ that does not correspond to a real sample. These centroids are sent to the server to determine which groups are selected for tuning.

在获得本地数据 $\mathcal{D}_i$ 的融合特征 (记为 $\{\tilde{\mathbf{h}}_1, \tilde{\mathbf{h}}_2, \dots, \tilde{\mathbf{h}}_{|\mathcal{D}_i|}\}$) 后，应用 HDBSCAN (Campello 等人, 2013) 将其聚类为若干组 $\{\mathcal{G}_1, \mathcal{G}_2, \dots\}$ 。每个组 $\mathcal{G}_j$ 都有一个并不对应于真实样本的质心 $\mathbf{c}_j$ 。这些质心被发送到服务器，以确定选择哪些组进行微调。

4.3 Inter-Client Data Selection

4.3 客户端间的数据选择

As discussed in Section 3 and shown in Figure 5, there may be similarities among client-side data. Thus, we cluster the approximate centroids sent from the clients to the server with HDBSCAN to filter redundant data groups among clients, as

正如第 3 节所述及图 5 所示，客户端数据之间可能存在相似性。因此，我们使用 HDBSCAN 对从客户端发送到服务器的近似质心进行聚类，以过滤客户端之间的冗余数据组，如下所示：

$$\mathbb{G}^{\Pi} = \{\mathcal{G}_1^{\Pi}, \mathcal{G}_2^{\Pi}, \dots\} = \text{HDBSCAN}(\{\mathbf{c}_1, \mathbf{c}_2, \dots\}).$$

For each $\mathcal{G}_j^{\Pi}$ with its approximate centroid $\mathbf{c}_j^{\Pi}$, the server identifies the first-layer group $\mathcal{G}_s$ whose centroid $\mathbf{c}_s$ closest to $\mathbf{c}_j^{\Pi}$ as the selected one, as

对于每个带有近似质心 $\mathbf{c}_j^{\Pi}$ 的 $\mathcal{G}_j^{\Pi}$ ，服务器将识别出其质心 $\mathbf{c}_s$ 距离 $\mathbf{c}_j^{\Pi}$ 最近的第一层组 $\mathcal{G}_s$ 作为被选中的组，如下所示：

$$\mathbb{G}^{\text{selected}} = \left\{ \mathcal{G}_s \mid \mathbf{c}_s = \arg \min_{\mathbf{c} \in \mathcal{G}_j^{\Pi}} \|\mathbf{c} - \mathbf{c}_j^{\Pi}\| \right\}_{\mathcal{G}_j^{\Pi} \in \mathbb{G}^{\Pi}}.$$

Then, the corresponding clients will be notified of the selection of first-layer groups. Given the partitioned data groups $\mathbb{G}_i = \{\mathcal{G}_1, \mathcal{G}_2, \dots\}$ of client i , the subset finally used for tuning $\mathcal{D}_i$ is obtained as:

随后，相应的客户端将收到第一层分组选择的通知。给定客户端 i 的分区数据组 $\mathbb{G}_i = \{\mathcal{G}_1, \mathcal{G}_2, \dots\}$ ，最终用于微调的子集 $\mathcal{D}_i$ 获取方式如下：

$$\tilde{\mathcal{D}}_i = \left\{ \mathbf{x} \mid q = \arg \min_{\mathbf{x} \in \mathcal{G}_j} \|\mathbf{x} - \mathbf{c}_j\| \right\}_{\mathcal{G}_j \in \mathbb{G}_i, \mathcal{G}_j \in \mathbb{G}^{\text{selected}}}$$

In each selected group $\mathcal{G}_j$, the data sample closest to centroid $\mathbf{c}_j$ is added in $\tilde{\mathcal{D}}_i$ ($\mathbf{x} \rightarrow \mathbf{c}_j$ in Figure 3).

在每个选定的组 $\mathcal{G}_j$ 中，最接近质心 $\mathbf{c}_j$ 的数据样本将被添加到 $\tilde{\mathcal{D}}_i$ ($\mathbf{x} \rightarrow \mathbf{c}_j$ 中 (见图 3))。

4.4 Instruction Tuning with Coresets

4.4 使用核心集进行指令微调

After determining coresets, each client i performs local training using only $\tilde{\mathcal{D}}_i$ (II in Figure 3), as

在确定核心集后，每个客户端 i 仅使用 $\tilde{\mathcal{D}}_i$ 进行本地训练 (见图 3 中的 II)，如下所示：

$$\mathbf{w}_{i,t+1}^r = \mathbf{w}_{i,t}^r - \eta \cdot \nabla_{\mathbf{w}_{i,t}^r} \mathcal{L}(\mathbf{w}_t^r; \mathbf{x}), \forall \mathbf{x} \in \tilde{\mathcal{D}}_i,$$

followed by sending the optimized model parameters. Then, the server performs FedAvg (McMahan et al., 2017) on the received parameters, as

随后发送优化后的模型参数。接着，服务器对接收到的参数执行 FedAvg (McMahan et al., 2017)，如下所示：

$$\mathbf{w}^{r+1} = \frac{1}{\sum_{i \in \mathbb{M}_r} |\tilde{\mathcal{D}}_i|} \sum_{i \in \mathbb{M}_r} |\tilde{\mathcal{D}}_i| \cdot \mathbf{w}_{i,-1}^r,$$

where $\mathbb{M}_r$ contains the indices of activate clients in the r -th round of FL. Finally, the next round starts.

其中 $\mathbb{M}_r$ 包含第 r 轮联邦学习 (FL) 中活跃客户端的索引。最后，开始下一轮。

FedHDS only performs downsampling on the local training data based on FedAvg, thus its convergence is theoretically supported, as discussed in Appendix B. Besides, we provide discussions on the theoretical speedup of FedHDS in Appendix

C.

FedHDS 仅在 FedAvg 的基础上对本地训练数据进行下采样，因此其收敛性在理论上是有支持的，详见附录 B。此外，我们在附录 C 中讨论了 FedHDS 的理论加速效果。

4.5 Further Enhancement for FedHDS

4.5 FedHDS 的进一步增强

Efficiency The efficiency of FedHDS can be further improved by faster feature extraction, since it requires forward propagation across all the local data with the LLM w . Motivated by studies on retrieval augmented generation which adopt a lightweight retrieval model to extract data features (Fan et al., 2024), we use a smaller language model sharing Transformer architecture and next-token prediction paradigm with w as a proxy to generate representations for each data, i.e., a small version of GPT-2 (about 124 M parameters). We term this approach as FedHDS-Turbo.

效率 FedHDS 的效率可以通过更快的特征提取进一步提升，因为它需要使用 LLM w 对所有本地数据进行前向传播。受检索增强生成研究的启发，即采用轻量级检索模型来提取数据特征 (Fan et al., 2024)，我们使用一个与 w 共享 Transformer 架构和下文预测范式的较小语言模型作为代理，为每个数据生成表示，例如小版本的 GPT-2 (约 1.24 亿参数)。我们将这种方法称为 FedHDS-Turbo。

Privacy Although local centroids sent to the server are two-dimensional vectors that do not correspond to real samples, FedHDS offers more information than vanilla FL. We can adopt a straightforward differential privacy method for scenarios requiring stronger privacy protection (Hu et al., 2022b). We scale elements in local centroids to $[-1, 1]$ with tanh to maintain data distinguishability despite extreme values. Gaussian noise is then added to the scaled centroids before transmission, deriving their differential privacy (Theorem 1). Experiments in Section 5.7 show that FedHDS performs well with a reasonable noise scale.

隐私 尽管发送到服务器的本地质心是二维向量，并不对应真实样本，但 FedHDS 提供了比原始联邦学习 (FL) 更多的信息。对于需要更强隐私保护的场景，我们可以采用简单的差分隐私方法 (Hu et al., 2022b)。我们使用 tanh 将本地质心中的元素缩放到 $[-1, 1]$ ，以在存在极端值的情况下保持数据可区分性。然后在传输前向缩放后的质心添加高斯噪声，从而推导出它们的差分隐私 (定理 1)。第 5.7 节的实验表明，FedHDS 在合理的噪声规模下表现良好。

Theorem 1. Let $\mathbf{c} \in [-1, 1]$ be the original centroid and $\mathbf{c}' = \mathbf{c} + \mathbf{z}$ be the one noised by Gaussian noise $\mathbf{z} \sim \mathcal{N}(0, \sigma^2 \mathbf{I})$ with standard deviation σ . Suppose $\epsilon, \delta \in (0, 1)$. Noised centroid $\mathbf{c}'$ satisfies (ϵ, δ) -differential privacy if $\sigma \geq \frac{2\sqrt{2\log(1.25/\delta)}}{\epsilon}$.

定理 1. 设 $\mathbf{c} \in [-1, 1]$ 为原始质心， $\mathbf{c}' = \mathbf{c} + \mathbf{z}$ 为经标准差为 σ 的高斯噪声 $\mathbf{z} \sim \mathcal{N}(0, \sigma^2 \mathbf{I})$ 污染后的质心。假设 $\epsilon, \delta \in (0, 1)$ 。如果 $\sigma \geq \frac{2\sqrt{2\log(1.25/\delta)}}{\epsilon}$ ，则加噪质心 $\mathbf{c}'$ 满足 (ϵ, δ) -差分隐私。

Proof. The proof follows directly by applying the Gaussian mechanism (Dwork et al., 2014), where the sensitivity (i.e., range of $\mathbf{c}$) is 2.

证明。 该证明直接应用高斯机制 (Dwork et al., 2014) 得出，其中敏感度 (即 $\mathbf{c}$ 的范围) 为 2。

5 Evaluations 5 评估

5.1 Experimental Setup 5.1 实验设置

Baselines We introduce six federated tuning methods using full data as baselines: 1) FedAvg that tunes and transmits the full LLM, included for reference due to high costs; 2&3) FedPTuning and FedPrompt (Kuang et al., 2024) that apply PEFT techniques of P-Tuning (Liu et al., 2023) and Prompt Tuning (Lester et al., 2021) based on FedAvg, respectively, trained with Adam (Kingma and Ba, 2015) optimizer; 4&5) FedIT (Zhang et al., 2024): instruction tuning based on FedAvg with

LoRA, optimized with Adam or SGD (FedITSGD); 6) FlexLoRA (Bai et al., 2024) that supports

基准方法 我们引入了六种使用全量数据的联邦微调方法作为基准：1) FedAvg：对全量 LLM 进行微调和传输，因成本极高，仅作参考；2&3) FedPTuning 和 FedPrompt (Kuang et al., 2024)：分别在 FedAvg 基础上应用 P-Tuning (Liu et al., 2023) 和 Prompt Tuning (Lester et al., 2021) 等 PEFT 技术，并使用 Adam (Kingma and Ba, 2015) 优化器进行训练；4&5) FedIT (Zhang et al., 2024)：基于 FedAvg 结合 LoRA 进行指令微调，使用 Adam 或 SGD (FedITSGD) 进行优化；6) FlexLoRA (Bai et al., 2024)：在 FedIT 基础上支持

LoRA adapters with varying ranks based on FedIT.

具有不同秩 (rank) 的 LoRA 适配器。

Given the lack of data-efficient works in FL, we develop two federated methods and a centralized one using coresets: 1) Random: It randomly selects a ratio of local data. Although being native, it is a strong baseline (Lin et al., 2024; Sachdeva et al., 2024) by preserving the original data distributions; 2) Perplexity that selects data with lower perplexity scores (Chen et al., 2024); and 3) Coreset-Cent (Chen et al., 2023) that selects a fixed ratio of data by K-means clustering on the last-layer features.

鉴于联邦学习 (FL) 领域缺乏数据高效利用的相关研究，我们开发了两种联邦方法和一种基于核心集 (Coresets) 的 centralized 方法：1) Random (随机)：随机选择一定比例的本地数据。尽管这种方法很朴素，但由于保留了原始数据分布，它是一个强有力的基准 (Lin et al., 2024; Sachdeva et al., 2024)；2) Perplexity (困惑度)：选择困惑度得分较低的数据 (Chen et al., 2024)；3) Coreset-Cent (Chen et al., 2023)：通过对最后一层特征进行 K-means 聚类，选择固定比例的数据。

Datasets and Evaluations Following Qin et al. (2024a); Kuang et al. (2024), we conduct experiments on Natural Instructions (Wang et al., 2022) (NI) and Dolly-15K (Conover et al., 2023), and employ Rouge-L on held-out tasks as the evaluation metrics. After preprocessing (detailed in Appendix D.4), NI contains 738 training tasks, each of which is assigned to a unique client, providing non-IIDness with feature skew, and the natively provided 119 test tasks are used for evaluation. Dolly-15K contains 8 tasks. The last one is used for evaluation, and the rest are partitioned to 200 clients via Dirichlet distribution with α set to 0.5 and 5.0, respectively. Experiments on these two datasets provide scenarios where the client has hundreds and dozens of data samples, respectively.

数据集与评估：参考 Qin et al. (2024a) 和 Kuang et al. (2024)，我们在 Natural Instructions (Wang et al., 2022) (NI) 和 Dolly-15K (Conover et al., 2023) 上进行实验，并采用预留任务上的 Rouge-L 作为评估指标。经过预处理（详见附录 D.4），NI 包含 738 个训练任务，每个任务分配给一个唯一的客户端，从而提供了具有特征偏移的非独立同分布 (non-IID) 特性，并使用原生提供的 119 个测试任务进行评估。Dolly-15K 包含 8 个任务，最后一个用于评估，其余任务通过狄利克雷分布 (Dirichlet distribution) 分配给 200 个客户端， α 分别设置为 0.5 和 5.0。在这两个数据集上的实验分别模拟了客户端拥有数百条和数十条数据样本的场景。

Implementation This work targets cross-device FL, thus, 5% of the clients are randomly selected to participate in each round. Limited by space, the implementation is detailed in Appendix D.

实现细节：本研究针对跨设备联邦学习，因此每轮随机选择 5% 的客户端参与。受篇幅限制，具体实现细节见附录 D。

5.2 Comparison on Accuracy

5.2 准确率对比

We compare these methods in Table 2. The results of full-data FL methods are derived from Qin et al. (2024a) under the same settings, and those of others are obtained within the best hyperparameters.

我们在表 2 中对这些方法进行了比较。全量数据联邦学习 (FL) 方法的结果源自 Qin 等人 (2024a) 在相同设置下的研究，其他方法的结果则是在最佳超参数设置下获得的。

Comparison to Full-Data Methods From Table 2, FedHDS and FedHDS-Turbo outperform full-data FL baselines across the six scenarios with consumed data samples less than 1.5% of them. Particularly, on NI with DataJuicer-1.3B, FedHDS and FedHDS-Turbo relatively improve Rouge-L over FedIT-the practical full-data FL baseline achieving the best average

accuracy-by 19.5% and 16.3%, respectively. Averaged across the six scenarios, FedHDS-Turbo improves the Rouge-L score relative to FedIT by 10.72%. Besides, compared to FedIT, FedHDS achieves an average improvement of 4.26% and 4.96% in Rouge-L on

与全量数据方法的比较 从表 2 可以看出，FedHDS 和 FedHDS-Turbo 在六个场景中的表现均优于全量数据联邦学习基准，且消耗的数据样本量不足其 1.5%。特别是在使用 DataJuicer-1.3B 的 NI 数据集上，FedHDS 和 FedHDS-Turbo 相比于 FedIT（取得最佳平均准确率的实际全量数据联邦学习基准），Rouge-L 分数分别相对提升了 19.5% 和 16.3%。在六个场景的平均值上，FedHDS-Turbo 相比 FedIT 将 Rouge-L 分数提升了 10.72%。此外，与 FedIT 相比，FedHDS 在 Rouge-L 上实现了平均 4.26% 和 4.96% 的提升。

Table 2: Table 2: Rouge-L (%) comparisons. Parentheses indicate the ratio of consumed data samples compared to full-data methods. Each value is the average Rouge-L obtained in the last round of four runs with different random seeds. Coreset-Cent and FedAvg are introduced just as references as they are not practical to end devices. Bold and underlined numbers are the best and second-best values among approaches practical to cross-device FL, respectively.

表 2：Rouge-L (%) 比较。括号内表示与全量数据方法相比所消耗的数据样本比例。每个数值是四次不同随机种子运行在最后一轮获得的平均 Rouge-L。Coreset-Cent 和 FedAvg 仅作为参考引入，因为它们在终端设备上并不实用。加粗和带下划线的数字分别是适用于跨设备联邦学习的方法中的最佳值和次佳值。

Approach 方法	Natural Instructions (Meta Non-IID)		Dolly-15K ($\alpha = 0.5$)		Dolly-15K ($\alpha = 5.0$)	
	DataJuicer-1.3B	LLaMA-3B	DataJuicer-1.3B	LLaMA-3B	DataJuicer-1.3B	LLaMA-3B
Coreset-Cent	31.36 $\pm$ 0.80 (1.00%)	34.81 $\pm$ 0.90 (0.01%)	33.27 $\pm$ 0.33 (0.50%)	35.48 $\pm$ 1.08 (1.00%)	33.27 $\pm$ 0.33 (0.50%)	35.48 $\pm$ 1.08 (1.00%)
FedAvg	22.08 $\pm$ 1.52 (100%)	27.88 $\pm$ 0.75 (100%)	32.30 $\pm$ 1.23 (100%)	34.27 $\pm$ 0.45 (100%)	33.38 $\pm$ 1.43 (100%)	33.95 $\pm$ 0.79 (100%)
FedPTuning	19.61 $\pm$ 2.71 (100%)	25.41 $\pm$ 1.14 (100%)	23.98 $\pm$ 3.23 (100%)	30.30 $\pm$ 1.16 (100%)	25.33 $\pm$ 2.48 (100%)	29.08 $\pm$ 1.33 (100%)
FedPrompt	6.04 $\pm$ 0.12 (100%)	8.95 $\pm$ 2.47 (100%)	32.73 $\pm$ 0.87 (100%)	24.50 $\pm$ 4.78 (100%)	32.51 $\pm$ 1.31 (100%)	23.94 $\pm$ 4.15 (100%)
FedIT-SGD	19.40 $\pm$ 1.83 (100%)	28.14 $\pm$ 0.85 (100%)	27.23 $\pm$ 0.68 (100%)	29.28 $\pm$ 0.50 (100%)	27.28 $\pm$ 1.35 (100%)	29.19 $\pm$ 0.89 (100%)
FlexLoRA	23.19 $\pm$ 2.14 (100%)	28.86 $\pm$ 0.55 (100%)	29.81 $\pm$ 1.06 (100%)	32.84 $\pm$ 0.99 (100%)	29.17 $\pm$ 1.35 (100%)	32.18 $\pm$ 1.28 (100%)
FedIT	22.30 $\pm$ 0.42 (100%)	28.13 $\pm$ 0.50 (100%)	30.80 $\pm$ 0.98 (100%)	33.23 $\pm$ 1.51 (100%)	30.97 $\pm$ 0.43 (100%)	33.68 $\pm$ 1.07 (100%)
Random 随机	26.20 $\pm$ 1.71 (0.20%)	31.23 $\pm$ 1.37 (2.00%)	32.59 $\pm$ 0.10 (1.50%)	33.82 $\pm$ 0.82 (1.50%)	32.24 $\pm$ 0.43 (1.50%)	34.29 $\pm$ 0.85 (5.00%)
Perplexity 困惑度	24.45 $\pm$ 0.77 (5.00%)	30.49 $\pm$ 0.21 (5.00%)	32.73 $\pm$ 0.15 (1.50%)	33.71 $\pm$ 0.51 (1.50%)	32.24 $\pm$ 0.22 (1.50%)	33.88 $\pm$ 0.34 (5.00%)
FedHDS	26.64 $\pm$ 0.79 (0.20%) ◀  ▶	<u>32.32 $\pm$ 0.92</u> (0.15%)	33.38 $\pm$ 0.40 (0.82%) ◀  ▶	35.40 $\pm$ 0.78 (1.18%) ◀  ▶	33.70 $\pm$ 0.19 (0.88%) ◀  ▶	35.79 $\pm$ 0.43 (1.34%) ◀  ▶
FedHDS-Turbo	25.93 $\pm$ 0.75 (0.23%) ◀  ▶	32.93 $\pm$ 0.64 (0.22%)	<u>33.28 $\pm$ 0.44</u> (1.31%)	<u>35.01 $\pm$ 0.65</u> (1.26%)	<u>33.52 $\pm$ 0.20</u> (1.19%)	35.42 $\pm$ 0.29 (1.28%)

Table 3: Table 3: Comparisons on 1) client-side time (CTime) i.e., the time a client spends performing local computations during a round of FL, 2) the overall time consumption across all rounds (total time), and 3) the speedup ratio. Time is calculated with training using CPU+GPU due to limited GPU memory on edge devices.

表 3：在以下方面的对比：1) 客户端时间 (CTime)，即客户端在每一轮联邦学习 (FL) 期间进行本地计算所花费的时间；2) 所有轮次的总体时间消耗 (总时间)；以及 3) 加速比。由于边缘设备上的 GPU 显存有限，时间是基于 CPU+GPU 训练计算得出的。

Approach 方法	DataJuicer-1.3B on NI NI 数据集上的 DataJuicer-1.3B			LLaMA-3B on NI NI 数据集上的 LLaMA-3B			DataJuicer-1.3B on Dolly-15K DataJuicer-1.3B 在 Dolly-15K 上			LLaMA-3B on Dolly-15K LLaMA-3B 在 Dolly-15K 上		
	CTime 通信时间	Total Time 总时间	Speedup 加速比	CTime 通信时间	Total Time 总时间	Speedup 加速比	CTime 计算时间	Total Time 总时间	Speedup 加速比	CTime 计算时间	Total Time 总时间	Speedup 加速比
FedIT	489S	8D9H □ 8 天 9 小时 □	1×	811S	13D21H	1×	56.0S	9H20M	1×	114S	19H6M	1×
Random 随机	13.2S	5H24M□	37.2×	35.3 S	14H29M□	23.0×	6.5 S	43M25S □	12.89×	25.2 S 25.2	4H12M□	4.54×
Perplexity 困惑度	46.6S	19H9M□	10.5×	85S	1D11H□	9.51×	8.9 S 8.9	59M27S □ 59 分 27 秒 □	9.42×	27.5 S	4H34M□	4.18×
FedHDS	27.5 S	11H17M□	17.8×	40.7 S	16H42M□	19.9×	5.2 S 5.2	34M52S □ 34 分 52 秒 □	16.06×	14.6 S 14.6	2 H 25 M □ 2 小时 25 分 □	7.90×
FedHDS-Turbo	10.0S	4H7M□	48.8×	20.1S	8H15M□	40.4×	4.5 S 4.5	29M42S □ 29 分 42 秒 □	18.86×	17.2S	2 H 52 M □ 2 小时 52 分 □	6.66×

Dolly- 15 K when $\alpha = 0.5$ and $\alpha = 5.0$, respectively, indicating that FedHDS performs better when clientside data has certain similarity. In a cross-device FL scenario with a large scale of clients, it is common for different clients to have similar data distributions. These results demonstrate the effectiveness of our approaches for improving generalization.

Dolly-15K 在 $\alpha = 0.5$ 和 $\alpha = 5.0$ 时，分别表明 FedHDS 在客户端数据具有一定相似性时表现更好。在拥有大规模客户端的跨设备联邦学习（FL）场景中，不同客户端具有相似的数据分布是很常见的。这些结果证明了我们方法在提高泛化能力方面的有效性。

Comparison to Coreset Baselines Both FedHDS and FedHDS-Turbo outperform Random and Perplexity in 5 of the 6 scenarios. Only on NI with DataJuicer-1.3B, Random slightly outperforms FedHDS-Turbo. From Lin et al. (2024); Sachdeva et al. (2024), Random is a strong baseline as it preserves the data distribution. However, it is affected by data ratios (Cao et al., 2024), and determining the optimal data ratio requires extensive experimentation. Differently, FedHDS and FedHDS-Turbo can automatically determine an appropriate data ratio. Note that in 7 out of the 12 scenarios involving Random and Perplexity, the data ratios with the best Rouge-L are inspired by our approaches. Even so, our approaches outperform them in the vast majority of cases, highlighting the need for a well-designed data selection strategy.

与核心集（Coreset）基准方法的对比：在 6 个场景中的 5 个场景里，FedHDS 和 FedHDS-Turbo 的表现均优于 Random（随机选择）和 Perplexity（困惑度）。仅在 DataJuicer-1.3B 的 NI 数据集上，Random 略微优于 FedHDS-Turbo。根据 Lin et al. (2024) 和 Sachdeva et al. (2024) 的研究，Random 是一个强基准，因为它保留了数据分布。然而，它会受到数据比例的影响 (Cao et al., 2024)，而确定最佳数据比例需要大量的实验。不同的是，FedHDS 和 FedHDS-Turbo 可以自动确定合适的数据比例。值得注意的是，在涉及 Random 和 Perplexity 的 12 个场景中，有 7 个场景的最佳 Rouge-L 数据比例是受我们方法的启发而设定的。即便如此，我们的方法在绝大多数情况下仍优于它们，这凸显了精心设计的数据选择策略的必要性。

The centralized method, Coreset-Cent, surpasses our approaches on the complex dataset, NI, indicating room for further improvement in FL methods.

中心化方法 Coreset-Cent 在复杂数据集 NI 上超过了我们的方法，这表明联邦学习方法仍有进一步改进的空间。

5.3 时间效率对比

From Table 3, by reducing the number of local training steps, coreset methods improve efficiency over FedIT. FedHDS-Turbo achieves a higher speedup due to the fewer data samples required for better accuracy compared to other coreset FL baselines. Particularly, on NI with DataJuicer-1.3B, FedHDS-Turbo achieves a speedup of $48.8\times$ over FedIT. Since FedHDS-Turbo is comparable to FedHDS in accuracy, it may be more applicable. We provide a breakdown of time consumption in Appendix E. 4 to clearly present the time consumption of each step in our approach.

从表 3 可以看出，通过减少本地训练步数，核心集（coreset）方法比 FedIT 提高了效率。由于与其他核心集联邦学习（FL）基准方法相比，FedHDS-Turbo 只需更少的数据样本即可获得更高的准确率，因此它实现了更高的加速比。特别是在使用 DataJuicer-1.3B 的 NI 数据集上，FedHDS-Turbo 相比 FedIT 实现了 $48.8\times$ 的加速。由于 FedHDS-Turbo 在准确率上与 FedHDS 相当，它可能更具适用性。我们在附录 E.4 中提供了耗时明细，以清晰地展示我们方法中每个步骤的耗时情况。

5.4 Ablation Studies 5.4 消融实验

Two-Layer Selection To clarify the contributions of intra- and inter-client selections, we build two methods: 1) FedHDS $\ddagger$: Removes inter-client selection and groups data by last-layer features. 2) FedHDS $\dagger$: Uses only intra-client selection, selecting one data sample closest to the centroid in each

双层选择 为了阐明客户端内（intra-client）和客户端间（inter-client）选择的贡献，我们构建了两种方法：1) FedHDS $\ddagger$ ：移除客户端间选择，并根据最后一层特征对数据进行分组。2) FedHDS $\dagger$ ：仅使用客户端内选择，在每个分组中选择一个最接近质心的数据样本。

Table 4: Table 4: Rouge-L (%) comparisons for ablation studies, organized in the same manner of Table 2.

表 4：消融实验的 Rouge-L (%) 对比，组织方式与表 2 相同。

Approach	方法	Natural Instructions (Meta Non-IID)		Dolly-15K ($\alpha = 0.5$)		Dolly-15K ($\alpha = 5.0$)	
		Natural Instructions (元非独立同分布)					
		DataJuicer-1.3B	LLaMA-3B	DataJuicer-1.3B	LLaMA-3B	DataJuicer-1.3B	LLaMA-3B
FedHDS ‡		25.45 ± 0.64(1.62%)	28.77 ± 1.91(3.86%)	31.82 ± 0.56(3.71%)	33.66 ± 0.48(4.25%)	32.12 ± 1.66(3.64%)	33.81 ± 0.87(4.32%)
FedHDS †		24.48 ± 0.78(0.83%)	32.27 ± 0.12(1.51%)	32.52 ± 0.65(2.93%)	34.11 ± 0.94(3.97%)	32.79 ± 1.15(2.99%)	33.98 ± 1.36(4.33%)
FedHDS		26.64 ± 0.79(0.20%)	32.32 ± 0.92(0.15%)	33.38 ± 0.40(0.82%) 	35.40 ± 0.78(1.18%) 	33.70 ± 0.19(0.88%) 	35.79 ± 0.43(1.34%)
w/i PCA	包含 PCA	27.84 ± 0.99 (0.09%)	32.62 ± 0.62(0.08%)	33.12 ± 0.10(0.74%)	34.90 ± 0.52(1.86%)	33.06 ± 0.23(0.74%)	34.60 ± 1.08(2.12%)
w/i KPCA	包含 KPCA	26.28 ± 3.93 (0.10%)	28.95 ± 0.92(0.08%)	33.21 ± 0.77(0.72%)	34.80 ± 0.95(1.62%)	33.34 ± 0.36(0.75%)	35.11 ± 0.27(2.08%)
FedHDS-Turbo		25.93 ± 0.75(0.23%)	32.93 ± 0.64(0.22%) 	33.28 ± 0.44(1.31%)	35.01 ± 0.65(1.26%)	33.52 ± 0.20(1.19%)	35.42 ± 0.29(1.28%)
w/i PCA	使用 PCA	22.93 ± 2.96(0.21%)	29.99 ± 0.78(0.21%)	33.03 ± 0.26(1.40%)	35.35 ± 0.65(1.87%)	33.15 ± 0.21(0.66%)	35.21 ± 0.54(1.72%)
w/i KPCA	使用 KPCA	25.80 ± 2.78 (0.03%)	28.98 ± 2.33(0.20%)	32.86 ± 1.08(1.86%)	35.04 ± 0.73(1.69%)	33.40 ± 0.41(0.71%)	34.88 ± 1.25(1.92%)

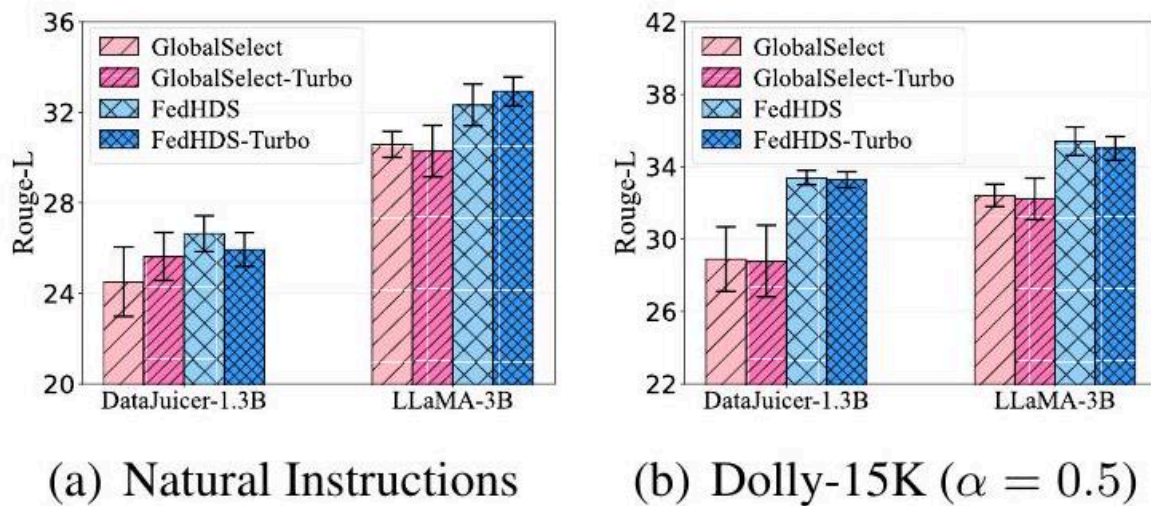


Figure 5: Figure 6: Performance of our approaches and methods that send data features to the server for global selection.

图 5: 图 6: 我们的方法与将数据特征发送到服务器进行全局选择的方法的性能对比。

local group. From Table 4, intra- and inter-client selections vary in effectiveness across different scenarios. FedHDS outperforms FedHDS * in all scenarios, showing the efficacy of the two selections.

本地组。从表 4 可以看出，客户端内选择和客户端间选择在不同场景下的效果各不相同。FedHDS 在所有场景下均优于 FedHDS *，证明了这两种选择策略的有效性。

To emphasize the necessity of hierarchical selection, we construct GlobalSelect, which sends all fused features to the server for global clustering, and GlobalSelect-Turbo that extracts features with GPT-2. From Figure 6, global selection may cause poor accuracy, potentially because clustering on the entire large-scale dataset yields suboptimal results. Thus, it is necessary to select data hierarchically.

为了强调分层选择的必要性，我们构建了 GlobalSelect（将所有融合特征发送到服务器进行全局聚类）以及 GlobalSelect-Turbo（使用 GPT-2 提取特征）。从图 6 可以看出，全局选择可能会导致准确率下降，这可能是因为在整个大规模数据集上进行聚类产生了次优结果。因此，进行分层数据选择是必要的。

Feature Fusion We explore the impact of the dimensionality reduction algorithm by replacing t-SNE with PCA (Jolliffe, 2002) and Kernel PCA (Schölkopf et al., 1997). From Table 4, on the relatively simpler datasets (Dolly- 15 K), replacing t-SNE brings minimal differences. However, on NI, substituting t-SNE occasionally results in negative effects. Thus, in practical scenarios, we recommend t-SNE for more effective coreset selection.

特征融合 我们通过将 t-SNE 替换为 PCA (Jolliffe, 2002) 和 Kernel PCA (Schölkopf et al., 1997) 来探索降维算法的影响。从表 4 可以看出，在相对较简单的数据集 (Dolly-15K) 上，替换 t-SNE 带来的差异极小。然而，在 NI 数据集上，替换 t-SNE 偶尔会产生负面影响。因此，在实际场景中，我们推荐使用 t-SNE 以实现更有效的核心集选择。

5.5 Communication and Memory Costs

5.5 通信与内存开销

We provide the maximum memory cost and clientside per-round communication cost of our approaches. Besides transmitting LoRA adapters as FedIT, i.e., Comm. (Model), FedHDS additionally

我们提供了所提方法的最高内存开销和客户端每轮通信开销。除了像 FedIT 那样传输 LoRA 适配器（即模型通信 Comm. (Model)）之外，FedHDS 还额外涉及

Table 5: Table 5: Per-round costs by 1.3 B models (Dolly-15K).

表 5：1.3 B 模型（Dolly-15K）的每轮成本。

	Comm. (Model) 通信（模型）	Comm. (Features & Cluster Indices) 通信（特征与聚类索引）	GPU Mem. GPU 显存
FedIT	12 MB	0	10.56 GB
FedHDS	12 MB	44 Bytes 44 字节	9.40 GB
FedHDS-Turbo	12 MB	76 Bytes 76 字节	9.32 GB

transmits 1) 2-dimensional cluster centroids and 2) indices of a few selected clusters. From Table 5, these bring negligible cost, i.e., just a few dozen bytes. Detailed calculations are in Appendix F.

传输 1) 二维聚类质心和 2) 少量选定聚类的索引。从表 5 可以看出，这些带来的成本微乎其微，即仅有几十个字节。详细计算见附录 F。

FedHDS’s memory usage is similar to FedIT, with a slight reduction from filtering long samples. Nevertheless, FedHDS makes computation offloading feasible for on-device LLM tuning by significantly reducing the required data samples, while training on the full dataset with enabling offloading incurs a substantial time cost (Figure 2).

FedHDS 的内存占用与 FedIT 相似，由于过滤了长样本，内存占用略有减少。尽管如此，FedHDS 通过显著减少所需的数据样本，使得设备端 LLM 微调的计算卸载（computation offloading）变得可行，而启用卸载并在全量数据集上进行训练则会产生巨大的时间成本（图 2）。

5.6 Studies on Convergence and Overfitting

5.6 收敛性与过拟合研究

To illustrate the convergence trends of these approaches, Figure 7 presents the convergence curves of FedHDS, FedHDS-Turbo, and the baseline methods using LLaMA-3B on Natural Instructions. The involved hyperparameters are aligned with those described in Section 5.1. It can be seen that these methods have nearly reached a convergent state by the 40th federated round, demonstrating that LLMs can be effectively tuned with a limited number of instruction data. Compared to approaches that use the full dataset, those relying on a data subset generally achieve a lower test loss. This is because tuning on the complete local dataset results in excessive local training steps, causing the LLM to overfit the local data and perform worse on unseen tasks, as discussed in Section 5.2.

为了说明这些方法的收敛趋势，图 7 展示了 FedHDS、FedHDS-Turbo 以及基线方法在 Natural Instructions 数据集上使用 LLaMA-3B 的收敛曲线。所涉及的超参数与第 5.1 节中描述的保持一致。可以看出，这些方法在第 40 个联邦轮次时已基本达到收敛状态，这表明利用有限数量的指令数据即可有效地微调 LLMs。与使用全量数据集的方法相比，依赖数据子集的方法通常能获得更低的测试损失。这是因为在完整的本地数据集上进行微调会导致过多的本地训练步骤，从而使 LLM 对本地数据产生过拟合，并在未见过的任务上表现较差，正如第 5.2 节中所讨论的那样。

To better demonstrate that training on a subset can alleviate the overfitting problem to some extent, we provide the training and test loss of FedHDS-

为了更好地证明在子集上训练可以在一定程度上缓解过拟合问题，我们提供了 FedHDS- 的训练和测试损失

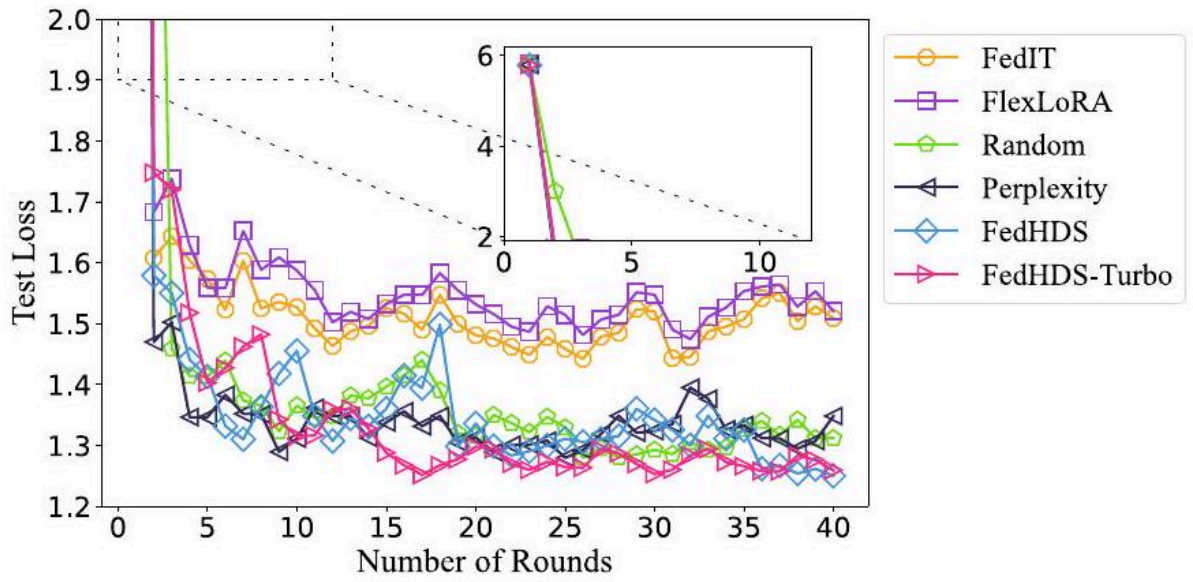


Figure 6: Figure 7: Convergence of the loss value on the test tasks obtained by FedHDS-Turbo and FedIT with LLaMA-3B on NI.

图 6：图 7：FedHDS-Turbo 与 FedIT 在 NI 数据集上使用 LLaMA-3B 得到的测试任务损失值收敛情况。

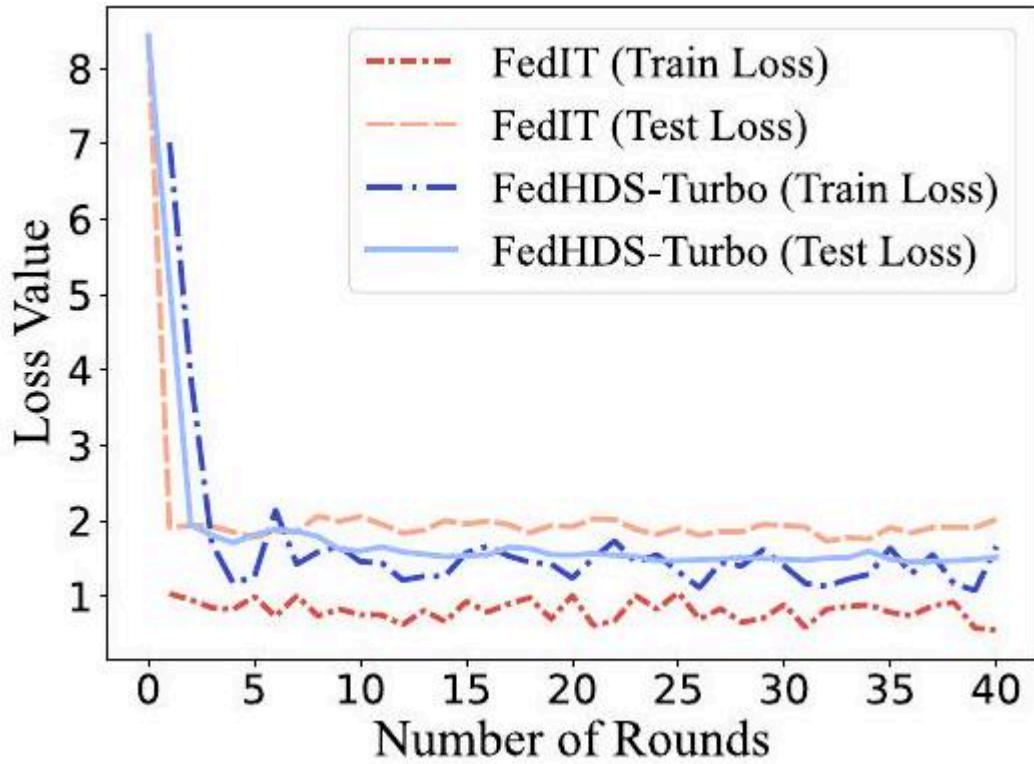


Figure 7: Figure 8: Convergence of training and test loss obtained by FedHDS-Turbo and FedIT with DataJuicer-1.3B on NI.

图 7：图 8：FedHDS-Turbo 与 FedIT 在 NI 数据集上使用 DataJuicer-1.3B 得到的训练和测试损失收敛情况。

Turbo with DataJuicer-1.3B on Natural Instructions in Figure 8, together with those of the federated approach using the full dataset that achieves the best average accuracy, i.e., FedIT. As shown in Figure 8, both the training and test loss of FedHDS-Turbo stably decrease. In contrast, the test loss of FedIT quickly stops decreasing while the training loss is still reducing to lower values. Finally, the test loss of FedHDS-Turbo is lower than that of FedIT, causing FedHDS-Turbo to

perform better than FedIT, as shown in Table 2.

图 8 展示了在 Natural Instructions 数据集上使用 DataJuicer-1.3B 的 Turbo 方法，以及使用全量数据集并达到最佳平均准确率的联邦学习方法（即 FedIT）的结果。如图 8 所示，FedHDS-Turbo 的训练损失和测试损失均稳定下降。相比之下，FedIT 的测试损失很快停止下降，而其训练损失仍在继续降低。最终，FedHDS-Turbo 的测试损失低于 FedIT，这使得 FedHDS-Turbo 的表现优于 FedIT，如表 2 所示。

From the above, we can conclude that FedHDS and FedHDS-Turbo exhibit stable convergence and perform on par with federated approaches that use the full dataset. Additionally, by reducing the amount of data used for instruction tuning, they mitigate overfitting to local data to some extent, leading to higher instruction-tuning accuracy compared to the baseline methods.

综上所述，我们可以得出结论：FedHDS 和 FedHDS-Turbo 表现出稳定的收敛性，且性能与使用全量数据集的联邦学习方法相当。此外，通过减少用于指令微调的数据量，它们在一定程度上缓解了对本地数据的过拟合，从而实现了比基准方法更高的指令微调准确率。

5.7 Performance with Differential Privacy

5.7 差分隐私下的性能

FedHDS can perform well with noise sampling variance no greater than 0.1 , which improves the privacy of client-side data. Limited by space, detailed experiments on this are left in Appendix E.3.

FedHDS 在噪声采样方差不大于 0.1 的情况下表现良好，这提高了客户端数据的隐私性。受限于篇幅，相关的详细实验内容保留在附录 E.3 中。

5.8 Performance in Various FL Scenarios

5.8 不同联邦学习场景下的性能

FedHDS outperforms coreset FL baselines with different active client ratios, showing its applicability in various FL scenarios. We demonstrate this with experimental results in Appendix E.5.

FedHDS 在不同活跃客户端比例下的表现均优于核心集联邦学习基准方法，展示了其在各种联邦学习场景中的适用性。我们在附录 E.5 的实验结果中证明了这一点。

6 Conclusion 6 结论

Existing federated instruction tuning methods for LLMs typically train LLMs using all local data, causing significant computation cost and overfitting to local data. This work pioneers an exploration into federated data-efficient instruction tuning, and

现有的 LLMs 联邦指令微调方法通常使用所有本地数据来训练 LLMs，这导致了巨大的计算成本以及对本地数据的过拟合。本工作开创性地探索了联邦数据高效指令微调，并且

proposes FedHDS, a coreset selection approach that solves both intra- and inter-client data redundancy. It fuses data features of varying abstraction levels obtained from different Transformer layers for better data representation. Extensive experiments involving various datasets, LLMs and nonIIDness demonstrate that FedHDS enhances the data efficiency and Rouge-L on unseen tasks over existing federated tuning methods.

提出了 FedHDS，这是一种解决客户端内和客户端间数据冗余的核心集选择方法。它融合了从不同 Transformer 层获取的不同抽象层级的数据特征，以实现更好的数据表示。涉及多种数据集、LLMs 和非独立同分布场景的广泛实验表明，与现有的联邦微调方法相比，FedHDS 提高了数据效率，并在未知任务上提升了 Rouge-L 指标。

7 Limitations 7 局限性

Although our approach improves data efficiency and generalization to unseen tasks in FL for LLM fine-tuning, it still has certain limitations. For example, it only selects data based on representativeness but overlooks data quality. Since domain divisions are usually implicit, low-quality data samples may also be treated as a separate domain. In this case, FedHDS may select low-quality data. Therefore, incorporating quality-based filtering mechanisms may help further improve our approach.

虽然我们的方法提高了 LLM 微调在联邦学习 (FL) 中的数据效率和对未知任务的泛化能力, 但仍存在一定的局限性。例如, 它仅基于代表性选择数据, 而忽略了数据质量。由于领域划分通常是隐性的, 低质量的数据样本也可能被视为一个独立的领域。在这种情况下, FedHDS 可能会选中低质量数据。因此, 引入基于质量的过滤机制可能有助于进一步改进我们的方法。

Acknowledgments 致谢

This research is supported in part by the National Science Foundation of China under Grants 62125206, the National Key Research and Development Program of China under Grant 2022YFB4500100, and the Zhejiang Provincial Natural Science Foundation of China under Grant LD24F020014. This research is also supported in part by the National Research Foundation, Singapore and Infocomm Media Development Authority under its Trust Tech Funding Initiative. Any opinions, findings and conclusions or recommendations expressed in this material are those of the author(s) and do not reflect the views of National Research Foundation, Singapore and Infocomm Media Development Authority.

本研究得到了中国国家自然科学基金 (项目编号: 62125206)、国家重点研发计划 (项目编号: 2022YFB4500100) 以及浙江省自然科学基金 (项目编号: LD24F020014) 的部分资助。本研究还得到了新加坡国家研究基金会和资讯通信媒体发展局在其信托科技资助计划 (Trust Tech Funding Initiative) 下的部分支持。本材料中表达的任何观点、发现、结论或建议均为作者个人观点, 不代表新加坡国家研究基金会和资讯通信媒体发展局的立场。

References 参考文献

- Sara Babakniya, Ahmed Roushdy Elkordy, Yahya H Ezzeldin, Qingfeng Liu, Kee-Bong Song, Mostafa El-Khamy, and Salman Avestimehr. 2023. SLoRA: Federated parameter efficient fine-tuning of language models. arXiv preprint arXiv:2308.06522.
- Jiamu Bai, Daoyuan Chen, Bingchen Qian, Liuyi Yao, and Yaliang Li. 2024. Federated fine-tuning of large language models under heterogeneous tasks and client resources. In The Thirty-eighth Annual Conference on Neural Information Processing Systems.
- Tadeusz Caliński and Jerzy Harabasz. 1974. A dendrite method for cluster analysis. *Communications in Statistics-theory and Methods*, 3(1):1-27.
- Ricardo JGB Campello, Davoud Moulavi, and Jörg Sander. 2013. Density-based clustering based on hierarchical density estimates. In Pacific-Asia conference on knowledge discovery and data mining, pages 160-172. Springer.
- Yihan Cao, Yanbin Kang, Chi Wang, and Lichao Sun. 2024. Instruction mining: Instruction data selection for tuning large language models. In First Conference on Language Modeling.
- Tianshi Che, Ji Liu, Yang Zhou, Jiayang Ren, Jiwen Zhou, Victor Sheng, Huaiyu Dai, and Dejing Dou. 2023. Federated learning of large language models with parameter-efficient prompt tuning and adaptive optimization. In Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, pages 7871-7888.
- Daoyuan Chen, Yilun Huang, Zhijian Ma, Hesun Chen, Xuchen Pan, Ce Ge, Dawei Gao, Yuexiang Xie, Zhaoyang Liu, Jinyang Gao, Yaliang Li, Bolin Ding, and Jingren Zhou. 2024. Data-juicer: A one-stop data processing system for large language models. In Companion of the 2024 International Conference on Management of Data, SIGMOD, pages 120-134. ACM.
- Hao Chen, Yiming Zhang, Qi Zhang, Hantao Yang, Xiaomeng Hu, Xuetao Ma, Yifan Yanggong, and Junbo Zhao. 2023. Maybe only 0.5% data is needed: A preliminary exploration of low training data instruction tuning. arXiv preprint arXiv:2305.09246.
- Mike Conover, Matt Hayes, Ankit Mathur, Jianwei Xie, Jun Wan, Sam Shah, Ali Ghodsi, Patrick Wendell, Matei Zaharia, and Reynold Xin. 2023. Free dolly: Introducing the world's first truly open instructiontuned LLM.
- Ganqu Cui, Lifan Yuan, Ning Ding, Guanming Yao, Wei Zhu, Yuan Ni, Guotong Xie, Zhiyuan Liu, and Maosong Sun. 2023. Ultrafeedback: Boosting language models with high-quality feedback. arXiv preprint arXiv:2310.01377.
- David L. Davies and Donald W. Bouldin. 1979. A cluster separation measure. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, PAMI-1(2):224-227.

Cynthia Dwork, Aaron Roth, et al. 2014. The algorithmic foundations of differential privacy. *Foundations and Trends® in Theoretical Computer Science*, 9(3-4):211-407.

Wenqi Fan, Yajuan Ding, Liangbo Ning, Shijie Wang, Hengyun Li, Dawei Yin, Tat-Seng Chua, and Qing Li. 2024. A survey on rag meeting llms : Towards retrieval-augmented large language models. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 64916501. ACM.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022a. Lora: Low-rank adaptation of large language models. In *The Tenth International Conference on Learning Representations, ICLR*. [OpenReview.net](https://openreview.net).

Hongsheng Hu, Zoran Salcic, Lichao Sun, Gillian Dobbie, Philip S Yu, and Xuyun Zhang. 2022b. Membership inference attacks on machine learning: A survey. *ACM Computing Surveys (CSUR)*, 54(11s):1-37.

Ian T Jolliffe. 2002. *Principal component analysis for special types of data*. Springer.

Diederik P. Kingma and Jimmy Ba. 2015. Adam: A method for stochastic optimization. In *3rd International Conference on Learning Representations, ICLR*.

Diederik P. Kingma 和 Jimmy Ba. 2015。Adam：一种随机优化方法。发表于第三届国际学习表征会议 (ICLR)。

Weirui Kuang, Bingchen Qian, Zitao Li, Daoyuan Chen, Dawei Gao, Xuchen Pan, Yuexiang Xie, Yaliang Li, Bolin Ding, and Jingren Zhou. 2024. Federatedscope11m : A comprehensive package for fine-tuning large language models in federated learning. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 5260-5271.

Weirui Kuang、Bingchen Qian、Zitao Li、Daoyuan Chen、Dawei Gao、Xuchen Pan、Yuexiang Xie、Yaliang Li、Bolin Ding 和 Jingren Zhou。2024。Federatedscope-llm：一个用于联邦学习中微调大语言模型的综合软件包。发表于第 30 届 ACM SIGKDD 知识发现与数据挖掘国际会议论文集，第 5260-5271 页。

Brian Lester, Rami Al-Rfou, and Noah Constant. 2021. The power of scale for parameter-efficient prompt tuning. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, EMNLP*, pages 3045-3059. Association for Computational Linguistics.

Brian Lester、Rami Al-Rfou 和 Noah Constant。2021。规模对于参数高效提示微调的力量。发表于 2021 年自然语言处理经验方法会议 (EMNLP) 论文集，第 3045-3059 页。计算语言学协会。

Ming Li, Lichang Chen, Jiuhai Chen, Shwai He, Jiuxiang Gu, and Tianyi Zhou. 2024a. Selective reflectiontuning: Student-selected data recycling for LLM instruction-tuning. In *Findings of the Association for Computational Linguistics, ACL 2024*, pages 1618916211.

Ming Li、Lichang Chen、Jiuhai Chen、Shwai He、Jiuxiang Gu 和 Tianyi Zhou。2024a。选择性反思微调：学生选择的数据循环用于 LLM 指令微调。发表于计算语言学协会调查结果 (ACL 2024)，第 16189-16211 页。

Ming Li, Yong Zhang, Shwai He, Zhitao Li, Hongyu Zhao, Jianzong Wang, Ning Cheng, and Tianyi Zhou. 2024b. Superfiltering: Weak-to-strong data filtering for fast instruction-tuning. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, ACL, pages 14255-14273. Association for Computational Linguistics.

第 14255-14273 页。Association for Computational Linguistics。

Ming Li, Yong Zhang, Zhitao Li, Jiuhai Chen, Lichang Chen, Ning Cheng, Jianzong Wang, Tianyi Zhou, and Jing Xiao. 2024c. From quantity to quality: Boosting LLM performance with self-guided data selection for instruction tuning. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, NAACL 2024, pages 7602-7635. Association for Computational

Ming Li, Yong Zhang, Zhitao Li, Jiahai Chen, Lichang Chen, Ning Cheng, Jianzong Wang, Tianyi Zhou, and Jing Xiao. 2024c. From quantity to quality: Boosting LLM performance with self-guided data selection for instruction tuning. In Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers), NAACL 2024, 第 7602-7635 页. Association for Computational Linguistics.

Xiang Li, Kaixuan Huang, Wenhao Yang, Shusen Wang, and Zhihua Zhang. 2020. On the convergence of fedavg on non-iid data. In International Conference on Learning Representations, ICLR. [OpenReview.net](#).

Xiang Li, Kaixuan Huang, Wenhao Yang, Shusen Wang, and Zhihua Zhang. 2020. On the convergence of fedavg on non-iid data. In International Conference on Learning Representations, ICLR. [OpenReview.net](#).

Xinyu Lin, Wenjie Wang, Yongqi Li, Shuo Yang, Fuli Feng, Yinwei Wei, and Tat-Seng Chua. 2024. Dataefficient fine-tuning for llm-based recommendation. In Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval, pages 365-374. ACM.

Zhenqing Ling, Daoyuan Chen, Liuyi Yao, Yaliang Li, and Ying Shen. 2024. On the convergence of zerothorder federated tuning for large language models. In Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, pages 1827-1838. ACM.

Xiao Liu, Yanan Zheng, Zhengxiao Du, Ming Ding, Yujie Qian, Zhilin Yang, and Jie Tang. 2023. GPT understands, too. AI Open, pages 208-215.

Keming Lu, Hongyi Yuan, Zheng Yuan, Runji Lin, Junyang Lin, Chuanqi Tan, Chang Zhou, and Jingren Zhou. 2023. #instag: Instruction tagging for analyzing supervised fine-tuning of large language models. In The Twelfth International Conference on Learning Representations. [OpenReview.net](#).

Sourab Mangrulkar, Sylvain Gugger, Lysandre Debut, Younes Belkada, Sayak Paul, and Benjamin Bossan. 2022. PEFT: State-of-the-art parameter-efficient finetuning methods. <https://github.com/huggingface/peft>.

Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas. 2017. Communication-efficient learning of deep networks from decentralized data. In the International Conference on Artificial Intelligence and Statistics, volume 54, pages 1273-1282.

Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. 2019. PyTorch: An imperative style, high-performance deep learning library. Advances in neural information processing systems, 32 .

Zhen Qin, Daoyuan Chen, Bingchen Qian, Bolin Ding, Yaliang Li, and Shuiguang Deng. 2024a. Federated full-parameter tuning of billion-sized language models with communication cost under 18 kilobytes. In Proceedings of the 41st International Conference on Machine Learning, ICML, volume 235, pages 41473-41497.

通信成本低于 18 KB 的十亿级语言模型全参数微调。收录于第 41 届国际机器学习大会 (ICML) 论文集, 第 235 卷, 第 41473-41497 页。

Zhen Qin, Daoyuan Chen, Wenhao Zhang, Liuyi Yao, Yilun Huang, Bolin Ding, Yaliang Li, and Shuiguang Deng. 2024b. The synergy between data and multi-modal large language models: A survey from co-development perspective. arXiv preprint arXiv:2407.08583.

Zhen Qin, Daoyuan Chen, Wenhao Zhang, Liuyi Yao, Yilun Huang, Bolin Ding, Yaliang Li, and Shuiguang Deng. 2024b. 数据与多模态大语言模型之间的协同效应: 协同开发视角下的综述. arXiv 预印本, arXiv:2407.08583.

Jeff Rasley, Samyam Rajbhandari, Olatunji Ruwase, and Yuxiong He. 2020. Deepspeed: System optimizations enable training deep learning models with over 100 billion parameters. In Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, pages 3505-3506.

Jeff Rasley, Samyam Rajbhandari, Olatunji Ruwase, and Yuxiong He. 2020. DeepSpeed: 通过系统优化实现超过 1000 亿参数的深度学习模型训练。收录于第 26 届 ACM SIGKDD 国际知识发现与数据挖掘大会论文集, 第 3505-3506 页。

Peter J Rousseeuw. 1987. Silhouettes: a graphical aid to the interpretation and validation of cluster analysis. Journal of computational and applied mathematics, 20:53-65.

Peter J Rousseeuw. 1987. 轮廓系数 (Silhouettes) : 聚类分析解释与验证的图形辅助方法. 计算与应用数学杂志 (Journal of computational and applied mathematics) , 20:53-65.

Noveen Sachdeva, Benjamin Coleman, Wang-Cheng Kang, Jianmo Ni, Lichan Hong, Ed H Chi, James Caverlee, Julian McAuley, and Derek Zhiyuan Cheng. 2024. How to train data-efficient llms. arXiv preprint arXiv:2402.09668.

Noveen Sachdeva, Benjamin Coleman, Wang-Cheng Kang, Jianmo Ni, Lichan Hong, Ed H Chi, James Caverlee, Julian McAuley, and Derek Zhiyuan Cheng. 2024. How to train data-efficient llms. arXiv preprint arXiv:2402.09668.

Bernhard Schölkopf, Alexander Smola, and KlausRobert Müller. 1997. Kernel principal component analysis. In International conference on artificial neural networks, pages 583-588. Springer.

Bernhard Schölkopf, Alexander Smola, and Klaus-Robert Müller. 1997. Kernel principal component analysis. In International conference on artificial neural networks, pages 583-588. Springer.

Youbang Sun, Zitao Li, Yaliang Li, and Bolin Ding. 2024. Improving lora in privacy-preserving federated learning. In The Twelfth International Conference on Learning Representations, ICLR. [OpenReview.net](https://openreview.net).

Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. 2023. Stanford Alpaca: An instruction-following llama model. https://github.com/tatsu-lab/stanford_alpaca.

Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. 2023. Stanford Alpaca: An instruction-following llama model. https://github.com/tatsu-lab/stanford_alpaca.

Laurens van der Maaten. 2013. Barnes-hut-sne. In 1st International Conference on Learning Representations, ICLR.

Laurens Van der Maaten and Geoffrey Hinton. 2008. Visualizing data using t -sne. Journal of machine learning research, 9(11):2579-2605.

Pablo Villalobos, Jaime Sevilla, Lennart Heim, Tamay Besiroglu, Marius Hobbhahn, and Anson Ho. 2022. Will we run out of data? an analysis of the limits of scaling datasets in machine learning. arXiv preprint arXiv:2211.04325.

Paul Voigt and Axel Von dem Bussche. 2017. The eu general data protection regulation (gdpr). A Practical Guide, 1st Ed., Cham: Springer International Publishing, 10(3152676):10-5555.

Paul Voigt and Axel Von dem Bussche. 2017. 欧盟通用数据保护条例 (GDPR). A Practical Guide, 1st Ed., Cham: Springer International Publishing, 10(3152676):10-5555.

Jiahao Wang, Bolin Zhang, Qianlong Du, Jiajun Zhang, and Dianhui Chu. 2024. A survey on data selection for llm instruction tuning. arXiv preprint arXiv:2402.05123.

Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A Smith, Daniel Khashabi, and Hannaneh Hajishirzi. 2023. Self-Instruct: Aligning language models with self-generated instructions. In Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 13484-13508.

Yizhong Wang, Swaroop Mishra, Pegah Alipoormolabashi, Yeganeh Kordi, Amirreza Mirzaei, Atharva Naik, Arjun Ashok, Arut Selvan Dhanasekaran, Anjana Arunkumar, David Stap, Eshaan Pathak, Giannis Karamanolakis, Haizhi Gary Lai, Ishan Purohit, Ishani Mondal, Jacob Anderson, Kirby Kuznia, Krithika Doshi, Kuntal Kumar Pal, Maitreya Patel, Mehrad Moradshahi, Mihir Parmar, Mirali Purohit, Neeraj Varshney, Phani Rohitha Kaza, Pulkit Verma, Ravsehaj Singh Puri, Rushang Karia, Savan Doshi, Shailaja Keyur Sampat, Siddhartha Mishra, Sujay Reddy A, Sumanta Patro, Tanay Dixit, and Xudong Shen. 2022. Super-naturalinstructions: Generalization via declarative instructions on 1600+ NLP tasks. In Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing, EMNLP, pages 5085-5109.

Jason Wei, Maarten Bosma, Vincent Y. Zhao, Kelvin Guu, Adams Wei Yu, Brian Lester, Nan Du, Andrew M. Dai, and Quoc V. Le. 2022. Finetuned language models are zero-shot learners. In The Tenth International Conference on Learning Representations, ICLR. [OpenReview.net](https://openreview.net).

Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, et al. 2020. Transformers: State-of-the-art natural language processing. In Proceedings of the 2020 conference on empirical methods in natural language processing: system demonstrations, pages 38-45.

Feijie Wu, Zitao Li, Yaliang Li, Bolin Ding, and Jing Gao. 2024. Fedbiot: Llm local fine-tuning in federated learning without full model. In Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, pages 3345-3355. ACM.

Shengguang Wu, Keming Lu, Benfeng Xu, Junyang Lin, Qi Su, and Chang Zhou. 2023. Self-evolved diverse data sampling for efficient instruction tuning. arXiv preprint arXiv:2311.08182.

Mengwei Xu, Dongqi Cai, Yaozong Wu, Xiang Li, and Shangguang Wang. 2024. FwdLLM: Efficient federated finetuning of large language models with perturbed inferences. In Proceedings of the 2024 USENIX Annual Technical Conference, USENIX ATC, pages 579-596.

Daochen Zha, Zaid Pervaiz Bhat, Kwei-Herng Lai, Fan Yang, Zhimeng Jiang, Shaochen Zhong, and Xia Hu. 2024. Data-centric artificial intelligence: A survey. ACM Computing Surveys, 57(5).

Daochen Zha, Zaid Pervaiz Bhat, Kwei-Herng Lai, Fan Yang, Zhimeng Jiang, Shaochen Zhong, and Xia Hu. 2024. 以数据为中心的人工智能：综述。ACM Computing Surveys, 57(5).

Jianyi Zhang, Saeed Vahidian, Martin Kuo, Chunyuan Li, Ruiyi Zhang, Tong Yu, Guoyin Wang, and Yiran Chen. 2024. Towards building the federatedgpt: Federated instruction tuning. In ICASSP 2024-2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), pages 6915-6919.

Jianyi Zhang, Saeed Vahidian, Martin Kuo, Chunyuan Li, Ruiyi Zhang, Tong Yu, Guoyin Wang, and Yiran Chen. 2024. 迈向构建 FederatedGPT：联邦指令微调。In ICASSP 2024-2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), pages 6915-6919.

Zhuo Zhang, Yuanhang Yang, Yong Dai, Qifan Wang, Yue Yu, Lizhen Qu, and Zenglin Xu. 2023. FedPETuning: When federated learning meets the parameter-efficient tuning methods of pre-trained language models. In Findings of the Association for Computational Linguistics: ACL, pages 9963-9977.

Zhuo Zhang, Yuanhang Yang, Yong Dai, Qifan Wang, Yue Yu, Lizhen Qu, and Zenglin Xu. 2023. FedPETuning：当联邦学习遇上预训练语言模型的参数高效微调方法。In Findings of the Association for Computational Linguistics: ACL, pages 9963-9977.

Chunting Zhou, Pengfei Liu, Puxin Xu, Srinivasan Iyer, Jiao Sun, Yuning Mao, Xuezhe Ma, Avia Efrat, Ping Yu, Lili Yu, et al. 2023. LIMA: Less is more for alignment. Advances in Neural Information Processing Systems, 36:55006-55021.

Chunting Zhou, Pengfei Liu, Puxin Xu, Srinivasan Iyer, Jiao Sun, Yuning Mao, Xuezhe Ma, Avia Efrat, Ping Yu, Lili Yu, et al. 2023. LIMA：对齐实验中少即是多。Advances in Neural Information Processing Systems, 36:55006-55021.

Appendix 附录

We provide more discussions and experiments of this work and organize them as follows:

我们为这项工作提供了更多的讨论和实验，并将其组织如下：

Table of Contents 目录
A Detailed Algorithm ... 13

A 详细算法 ... 13

B Convergence Analysis ... 13

B 收敛性分析 ... 13

C Analysis on Speedup Ratio ... 15

C 加速比分析 ... 15

D Reproducibility ... 15

D 可复现性 ... 15

D. 1 Experimental Environments for Accuracy Evaluation ... 15

D. 1 准确度评估的实验环境 ... 15

D. 2 Experimental Environments for Memory Footprint and Efficiency Statistics ... 15

D. 2 内存占用与效率统计的实验环境 ... 15

D. 3 Detailed Hyperparameters ... 15

D. 3 详细超参数 ... 15

D. 4 Detailed Descriptions on Datasets ... 16

D. 4 数据集详细说明 ... 16

E Additional Experiments ... 16

E 补充实验 ... 16

E. 1 Evaluation of Features from Different Transformer Layers ... 16

E. 1 不同 Transformer 层的特征评估 ... 16

E. 2 Virtualization of Data Features ... 16

E. 2 数据特征虚拟化 ... 16

E. 3 Performance with Differential Privacy ... 17

E. 3 差分隐私下的性能表现 ... 17

E. 4 Breakdown of Time Consumption ... 17

E. 4 耗时详情分解 ... 17

E. 5 Performance in Various FL Scenarios ... 18

E. 5 各种联邦学习场景下的性能 ... 18

F Detailed Calculation of Communication Overhead ... 19

F 通信开销的详细计算 ... 19

To facilitate a better understanding of each step of the proposed approach, we provide Algorithm 1 to explain how FedHDS selects the coresets for activate clients in each round r , where lines 2 ~ 4 and 7 ~ 9 are performed individually by each client, and lines 1 and 5 ~ 6 are performed by the server.

为了便于更好地理解所提方法的每个步骤，我们提供了算法 1 来解释 FedHDS 如何在每一轮 r 中为活跃客户端选择核心集，其中第 2 ~ 4 行和第 7 ~ 9 行由每个客户端独立执行，而第 1 行和第 5 ~ 6 行由服务器执行。

B Convergence Analysis B 收敛性分析

FedHDS shares the same global objective with FedAvg (FedIT) defined in Eq. (1). The essential difference between FedHDS and FedAvg (FedIT) is that it optimizes only on a subset of the original data, as illustrated by the difference between Eq. (11) and Eq. (2). Given the global objective

FedHDS 与等式 (1) 中定义的 FedAvg (FedIT) 具有相同的全局目标。FedHDS 与 FedAvg (FedIT) 的本质区别在于，它仅在原始数据的子集上进行优化，如等式 (11) 与等式 (2) 之间的差异所示。给定等式 (1) 中定义的 FedHDS 全局目标

of FedHDS defined in Eq. (1), we have the local objective of each client i during local training as:

，每个客户端 i 在本地训练期间的本地目标为：

$$\min_{\mathbf{w}} f_i(\mathbf{w}) \triangleq \frac{1}{|\tilde{\mathcal{D}}_i|} \sum_{i=1}^{|\tilde{\mathcal{D}}_i|} \mathbb{E}_{\mathbf{x} \sim \tilde{\mathcal{D}}_i} [\mathcal{L}(\mathbf{w}; \mathbf{x})]$$

Following Li et al. (2020), we first make the following assumptions:

参考 Li et al. (2020)，我们首先做出以下假设：

Assumption 1. The local objective of each client i is L -smooth, i.e.,
 $f_i(\mathbf{v}) \leq f_i(\mathbf{w}) + (\mathbf{v} - \mathbf{w})^T \nabla f_i(\mathbf{w}) + \frac{L}{2} \|\mathbf{v} - \mathbf{w}\|_2^2, \forall \mathbf{v} \in \mathbb{R}^d, \forall \mathbf{w} \in \mathbb{R}^d$.

假设 1. 每个客户端 i 的局部目标是 L -光滑的，即
 $f_i(\mathbf{v}) \leq f_i(\mathbf{w}) + (\mathbf{v} - \mathbf{w})^T \nabla f_i(\mathbf{w}) + \frac{L}{2} \|\mathbf{v} - \mathbf{w}\|_2^2, \forall \mathbf{v} \in \mathbb{R}^d, \forall \mathbf{w} \in \mathbb{R}^d$ 。

Assumption 2. The local objective of each client i is μ -convex, i.e.,
 $f_i(\mathbf{v}) \geq f_i(\mathbf{w}) + (\mathbf{v} - \mathbf{w})^T \nabla f_i(\mathbf{w}) + \frac{\mu}{2} \|\mathbf{v} - \mathbf{w}\|_2^2, \forall \mathbf{v} \in \mathbb{R}^d, \forall \mathbf{w} \in \mathbb{R}^d$.

假设 2. 每个客户端 i 的局部目标是 μ -强凸的，即
 $f_i(\mathbf{v}) \geq f_i(\mathbf{w}) + (\mathbf{v} - \mathbf{w})^T \nabla f_i(\mathbf{w}) + \frac{\mu}{2} \|\mathbf{v} - \mathbf{w}\|_2^2, \forall \mathbf{v} \in \mathbb{R}^d, \forall \mathbf{w} \in \mathbb{R}^d$ 。

Assumption 3. The variance of the stochastic gradients across each client is bounded, i.e.,
 $\mathbb{E} \|\nabla f_i(\mathbf{w}_{i,\tau}, \mathbf{x}) - \nabla f_i(\mathbf{w}_{i,\tau})\|^2 \leq \sigma_i^2$, where $\mathbf{w}_{i,\tau}$ denotes the model parameters of the i -th client after τ steps of updates.

假设 3. 每个客户端随机梯度的方差是有界的，即 $\mathbb{E} \|\nabla f_i(\mathbf{w}_{i,\tau}, \mathbf{x}) - \nabla f_i(\mathbf{w}_{i,\tau})\|^2 \leq \sigma_i^2$ ，其中 $\mathbf{w}_{i,\tau}$ 表示第 i 个客户端在经过 τ 步更新后的模型参数。

Assumption 4. The expected squared norm of the stochastic gradients stays within a uniform bound, i.e.,
 $\mathbb{E} \|\nabla f_i(\mathbf{w}_{i,\tau}, \mathbf{x})\|^2 \leq G^2$.

假设 4. 随机梯度的期望平方范数保持一致界限内，即 $\mathbb{E} \|\nabla f_i(\mathbf{w}_{i,\tau}, \mathbf{x})\|^2 \leq G^2$ 。

Existing works also make similar assumptions on the convergence analysis of federated tuning of LLMs (Ling et al., 2024). We also mildly assume that in each round, there are K clients on average that will submit their tuned models to the server,

and each of them performs E steps of local training to update their local models.

现有工作在 LLMs 联邦微调的收敛性分析中也做出了类似的假设 (Ling et al., 2024)。我们还温和地假设, 在每一轮中, 平均有 K 个客户端会将其微调后的模型提交给服务器, 并且每个客户端执行 E 步本地训练来更新其本地模型。

Theorem 2. Let Assumptions 1 $\sim$ 4 hold, $\mathbf{w}^*$ be the optimal global model, $\kappa = \frac{L}{\mu}, \gamma = \max\{8\kappa, E\}$, $B = \sum_{i=1}^N \lambda_i^2 \sigma_i^2 + 6LT + 8(E-1)^2 G^2$, $C = \frac{4}{K} E^2 G^2$, and $\mathbf{E} = \mathbb{E}[f(\mathbf{w}_T)] - f(\mathbf{w}^*)$, after T iterations, we have

定理 2. 假设 1 $\sim$ 4 成立, $\mathbf{w}^*$ 为最优全局模型, $\kappa = \frac{L}{\mu}, \gamma = \max\{8\kappa, E\}$, $B = \sum_{i=1}^N \lambda_i^2 \sigma_i^2 + 6LT + 8(E-1)^2 G^2$, $C = \frac{4}{K} E^2 G^2$ 且 $\mathbf{E} = \mathbb{E}[f(\mathbf{w}_T)] - f(\mathbf{w}^*)$, 在 T 次迭代后, 我们有

$$\mathbf{E} \leq \frac{2\kappa}{\gamma + T} \left(\frac{B + C}{\mu} + 2L \|\mathbf{w}^0 - \mathbf{w}^*\|^2 \right).$$

Proof. In fact, FedHDS is based on FedAvg and performs downsampling on local data, thereby affecting the number of local training iterations. With appropriate variable substitution, the convergence of FedHDS can be derived from the proof process in the work of Li et al. (2020).

证明. 事实上, FedHDS 基于 FedAvg 并对本地数据进行下采样, 从而影响本地训练迭代次数。通过适当的变量替换, FedHDS 的收敛性可以从 Li et al. (2020) 的证明过程中推导出来。

Based on Theorem 2, FedHDS has a convergence guarantee. Compared to FedIT, which directly adopts the training processes of FedAvg,

基于定理 2, FedHDS 具有收敛性保证。与直接采用 FedAvg 训练过程的 FedIT 相比,

Algorithm 1: Processes of Data Selection in FedHDS in each round (r) of FL.



```

Input:  $\{\mathbb{M}\}_r, \left\{ \mathcal{D}_{\{1\}}, \dots, \mathcal{D}_{\left| \mathbb{M} \right|} \right\}$ 
Output: The selected coreset  $\{\tilde{\mathcal{D}}_{\{i\}}\}$  for each client  $(i)$  ac
 $\left\{ \tilde{\mathcal{D}}_{\{1\}}, \dots, \tilde{\mathcal{D}}_{\left| \mathbb{M} \right|} \right\}$ 
Initialize a list  $\{\mathbb{C}\}$  to stage the received centroids
for  $(i=1, 2, \dots, \left| \mathbb{M} \right|)$  do
     $\left\{ \mathbf{c}_{\{1\}}, \mathbf{c}_{\{2\}}, \dots \right\} = \text{IntraClientSelection} \left( \mathbb{C} = \mathbb{C} \cup \left\{ \mathbf{c}_{\{1\}}, \mathbf{c}_{\{2\}}, \dots \right\} \right)$ 
     $\mathbb{G}^{\text{selected}} = \text{InterClientSelection} \left( \mathbb{C} \right)$ 
    send indices of selected groups to corresponding clients // (6) in Figure 3
    for  $(i=1, 2, \dots, \left| \mathbb{M} \right|)$  do
         $\tilde{\mathcal{D}}_{\{i\}} = \left\{ \mathbf{x} \mid q = \underset{\mathbf{x} \in \mathcal{D}_{\{i\}}}{\arg \min} \|\mathbf{x} - \mathbf{c}_{\{i\}}\| \right\}$ 
    return the selected subset  $\{\tilde{\mathcal{D}}_{\{i\}}\}$  for each client  $(i \in \mathbb{M})$ 
Function IntraClientSelection(  $\mathbf{w}, \mathcal{D}$  ):
    for  $(j=1, 2, \dots, |\mathcal{D}|)$  do
        extract features of  $\mathbf{x}_{\{j\}}$  by each layer of  $\mathbf{w}$ , denot
        // (1) in Figure 3
    reduce dimensionality of  $\mathbb{H} = \left\{ \mathbf{h}_{\{1\}}, \mathbf{h}_{\{2\}}, \dots, \mathbf{h}_{\left| \mathbb{H} \right|} \right\}$ 
     $\left\{ \tilde{\mathbf{h}}_{\{1\}}, \tilde{\mathbf{h}}_{\{2\}}, \dots, \tilde{\mathbf{h}}_{\left| \mathbb{H} \right|} \right\}$ 
    Figure 3

```

```

cluster data in  $\{\mathcal{D}_i\}$  based on their fused features, as
 $\left\{\mathcal{G}_1, \mathcal{G}_2, \dots\right\} = \text{operatorname{HDBSCAN}}$ 
approximate centroid  $\{\mathbf{c}_j\}$  // (3) in Figure 3
return  $\left\{\mathbf{c}_1, \mathbf{c}_2, \dots\right\}$ 
Function InterClientSelection  $\left(\left\{\mathbf{c}_1, \mathbf{c}_2, \dots\right\}\right)$ 
perform HDBSCAN to partition received  $\left\{\mathbf{c}_1, \mathbf{c}_2, \dots\right\}$ 
// (5) in Figure 3
initialize a list  $\mathcal{G}^{\text{selected}}$ 
for  $(j=1, 2, \dots, \left|\left\{\mathcal{G}_1^{\text{I}}, \mathcal{G}_2^{\text{I}}, \dots\right\}\right|)$ 
 $(s = \underset{\mathbf{c} \in \mathcal{G}_j^{\text{I}}}{\arg \min} \left\{\right.$ 
add  $\mathcal{G}_s$  into  $\mathcal{G}^{\text{selected}}$ 
return  $\mathcal{G}^{\text{selected}}$ 

```

the convergence rate of FedHDS on the training set theoretically has certain disadvantages, which have been experimentally demonstrated in Figure 7. However, the advantage of FedHDS is its ability to handle overfitting to local data. As shown in both Figures 7 and 8, FedHDS and FedHDS-Turbo significantly outperform FedIT in terms of test loss. Therefore, FedHDS and FedHDS-Turbo achieve better Rouge-L on held-out tasks than FedIT, as presented in Table 2.

从理论上讲，FedHDS 在训练集上的收敛速度存在一定的劣势，这在图 7 中已通过实验得到证实。然而，FedHDS 的优势在于其处理局部数据过拟合的能力。如图 7 和图 8 所示，FedHDS 和 FedHDS-Turbo 在测试损失方面显著优于 FedIT。因此，如表 2 所示，FedHDS 和 FedHDS-Turbo 在预留任务上取得了比 FedIT 更好的 Rouge-L 结果。

C Analysis on Speedup Ratio

C 加速比分析

In this section, we provide a brief analysis of the speedup achievable by our approaches to better understand their effectiveness in acceleration based on numerical results in Table 3.

在本节中，我们根据表 3 中的数值结果，对我们方法所能实现的加速效果进行简要分析，以更好地理解其在加速方面的有效性。

For a client with M data samples, the time complexity of performing t-SNE with Barnes-Hut implementation is $\mathcal{O}(M \log M)$, and that of performing HDBSCAN is $\mathcal{O}(M \log M) \sim \mathcal{O}(M^2)$ based on data sparsity (we adopt the worst complexity). Assume that ν and ϵ are the scale constants between the actual time consumption and complexity of t-SNE and HDBSCAN, respectively, and ξ and Ξ denote the time consumption (e.g., seconds) of performing one-step inference and training with one data sample, respectively. Assuming that the proportion of training data that FedHDS can filter out is ς , the time consumption of a client by conducting LLM instruction tuning with FedHDS is

对于拥有 M 个数据样本的客户端，使用 Barnes-Hut 实现进行 t-SNE 的时间复杂度为 $\mathcal{O}(M \log M)$ ，而根据数据稀疏性，执行 HDBSCAN 的时间复杂度为 $\mathcal{O}(M \log M) \sim \mathcal{O}(M^2)$ （我们采用最坏情况下的复杂度）。假设 ν 和 ϵ 分别是 t-SNE 和 HDBSCAN 的实际耗时与复杂度之间的比例常数， ξ 和 Ξ 分别表示对单个数据样本进行单步推理和训练的耗时（例如秒）。假设 FedHDS 可以过滤掉的训练数据比例为 ς ，则客户端使用 FedHDS 进行 LLM 指令微调的耗时为

$$\nu \cdot M \log M + \epsilon \cdot M^2 + \xi \cdot M + \Xi(1 - \varsigma)M$$

while that of FedIT, the approach using full data, is $\Xi \cdot M$. Therefore, the speeding-up ratio can be formalized as:

而使用全部数据的 FedIT 方法的时间为 $\Xi \cdot M$ 。因此，加速比可以公式化为：

$$\frac{\Xi \cdot M}{\nu \cdot M \log M + \epsilon \cdot M^2 + \xi \cdot M + \Xi(1-\varsigma)M}.$$

For these notations: 对于这些符号：

Generally, $\Xi > \xi$ by several times.

通常情况下， $\Xi > \xi$ 会提高数倍。

From the experiments on Dolly- 15 K, t-SNE on 200 samples takes 0.5 seconds, and HDBSCAN takes only 0.003 seconds. Based on these statistics, ν could be in the order of 10^{-4} , and ϵ could be in the order of 10^{-8} . Therefore, $\xi \gg \nu$ and $\xi \gg \epsilon$.

根据在 Dolly- 15 K, t-SNE 上对 200 个样本进行的实验，耗时为 0.5 秒，而 HDBSCAN 仅需 0.003 秒。基于这些统计数据， ν 的数量级可能为 10^{-4} ，而 ϵ 的数量级可能为 10^{-8} 。因此， $\xi \gg \nu$ 且 $\xi \gg \epsilon$ 。

ς can exceed 99% on Natural Instructions.

ς 在 Natural Instructions 数据集上可以超过 99%。

Therefore, the speedup achieved by our approaches is significant, as experimentally demonstrated in Table 3. Besides, the inference can be significantly accelerated (by reducing ξ), leading to a considerable improvement, e.g., FedHDS-Turbo is significantly faster than FedHDS.

因此，我们的方法所实现的加速效果非常显著，如表 3 中的实验所示。此外，推理过程也可以得到显著加速（通过减少 ξ ），从而带来可观的提升，例如 FedHDS-Turbo 明显快于 FedHDS。

D Reproducibility D 可复现性

D. 1 Experimental Environments for Accuracy Evaluation

D. 1 准确度评估的实验环境

We implement these approaches mentioned above with PyTorch (Paszke et al., 2019) 2.0.1, Transformers (Wolf et al., 2020) 4.31.0, scikit-learn 1.5.1, PEFT (Mangrulkar et al., 2022) 0.4.0, and hdbscan 0.8.37. Numerical experiments in Tables 2 and 4 are performed on platforms equipped with NVIDIA A100 or NVIDIA A6000 GPUs, installed with Python 3.10 and CUDA 12.4. Efficiency results in Table 3 are obtained on a platform with an NVIDIA A6000 GPU, installed with Python 3.10, CUDA 12.4 and DeepSpeed 0.15.2.

我们使用 PyTorch (Paszke et al., 2019) 2.0.1、Transformers (Wolf et al., 2020) 4.31.0、scikit-learn 1.5.1、PEFT (Mangrulkar et al., 2022) 0.4.0 以及 hdbscan 0.8.37 实现了上述方法。表 2 和表 4 中的数值实验是在配备 NVIDIA A100 或 NVIDIA A6000 GPU 的平台上进行的，安装了 Python 3.10 和 CUDA 12.4。表 3 中的效率结果是在配备 NVIDIA A6000 GPU 的平台上获得的，安装了 Python 3.10、CUDA 12.4 和 DeepSpeed 0.15.2。

D. 2 Experimental Environments for Memory Footprint and Efficiency Statistics

D. 2 显存占用与效率统计的实验环境

The memory footprint and time consumption in Figures 1 and 2 are measured with a maximum token list length of 1,024 where excessively long data will be truncated. The adopted platform is equipped with an NVIDIA A6000 GPU, installed with Python 3.10, CUDA 12.4 and DeepSpeed 0.15.2. For the memory footprint, the 95th percentile is calculated. The selected two GPUs are based on the most popular desktop and laptop GPUs identified from the Steam Hardware Survey

(Dec 2024).

图 1 和图 2 中的显存占用和时间消耗是在最大 Token 列表长度为 1,024 的情况下测量的，过长的数据将被截断。采用的平台配备了 NVIDIA A6000 GPU，安装了 Python 3.10、CUDA 12.4 和 DeepSpeed 0.15.2。对于显存占用，计算了第 95 百分位数。所选的两款 GPU 是根据 Steam 硬件调查（2024 年 12 月）确定的最受欢迎的台式机和笔记本电脑 GPU。

D. 3 Detailed Hyperparameters

D. 3 详细超参数

In approaches involving the LoRA adapter, i.e., FedHDS, FedHDS-Turbo, FedIT and FlexLoRA, the adapters are configured with the same hyperparameter settings, i.e., rank, alpha and dropout of LoRA adapters are set to 8, 16 and 0.05 for FedHDS and FedHDS-Turbo, respectively. Note that although FlexLoRA supports heterogeneous-rank LoRA adapters, we adopt a homogeneous-rank setting to ensure a fair comparison. Coreset-based methods perform 60 rounds of FL on Dolly-15 K with LLaMA-3B, and 40 rounds for other scenarios, where the local training is performed on the coreset for one epoch with Adam optimizer. The learning rate and number of rounds for federated

在涉及 LoRA 适配器的方法（即 FedHDS、FedHDS-Turbo、FedIT 和 FlexLoRA）中，适配器均配置了相同的超参数设置，即 FedHDS 和 FedHDS-Turbo 的 LoRA 适配器秩（rank）、alpha 和丢弃率（dropout）分别设置为 8、16 和 0.05。请注意，尽管 FlexLoRA 支持异构秩的 LoRA 适配器，但为了确保公平比较，我们采用了同构秩设置。基于核心集（Coreset）的方法在 LLaMA-3B 模型的 Dolly-15K 数据集上进行 60 轮联邦学习（FL），在其他场景下进行 40 轮，其中本地训练在核心集上使用 Adam 优化器执行一个轮次（epoch）。使用全量数据的联邦学习方法的学习率和轮数

approaches using full data are aligned with those in the work of Qin et al. (2024a).

与 Qin 等人 (2024a) 的工作保持一致。

We perform a preliminary hyperparameter search for federated approaches using coresets, and adopt the advantageous settings for each approach in each scenario for large-scale experiments. Specifically, we first search the learning rate in $\{3 \times 10^{-4}, 1 \times 10^{-4}, 3 \times 10^{-5}\}$. Then, for Random and Perplexity, we search the ratio of data samples in the final selected subsets to the full data samples in $\{0.2\%, 1.5\%, 2\%, 5\%\}$. Note that the thresholds of 0.2% and 1.5% are inspired by the proportion automatically obtained by FedHDS and FedHDS-Turbo. Considering the importance of an appropriate data ratio (Cao et al., 2024), these two baselines have benefited to some extent from the data proportion provided by FedHDS and FedHDS-Turbo. The finally adopted values for Tables 2 and 4 are as follows: All the federated approaches with coresets adopt a learning rate η of 3×10^{-5} on Dolly-15K. On Natural Instructions, FedHDS and FedHDS-Turbo adopt $\eta = 3 \times 10^{-4}$, Random adopt $\eta = 3 \times 10^{-4}$ with DataJuicer-1.3B and $\eta = 3 \times 10^{-5}$ with LLaMA-3B, Perplexity adopt $\eta = 3 \times 10^{-5}$ with DataJuicer-1.3B and $\eta = 3 \times 10^{-4}$ with LLaMA-3B.

我们对使用核心集的联邦学习方法进行了初步的超参数搜索，并在大规模实验中为每个场景下的每种方法采用了最优设置。具体而言，我们首先在 $\{3 \times 10^{-4}, 1 \times 10^{-4}, 3 \times 10^{-5}\}$ 中搜索学习率。然后，对于 Random 和 Perplexity 方法，我们在 $\{0.2\%, 1.5\%, 2\%, 5\%\}$ 中搜索最终选定子集的数据样本占全量数据样本的比例。需要注意的是，0.2% 和 1.5% 的阈值灵感来源于 FedHDS 和 FedHDS-Turbo 自动获取的比例。考虑到合适的比例的重要性（Cao 等人，2024），这两个基准方法在一定程度上受益于 FedHDS 和 FedHDS-Turbo 提供的比例。表 2 和表 4 最终采用的数值如下：所有使用核心集的联邦学习方法在 Dolly-15K 上均采用 3×10^{-5} 的学习率 η 。在 Natural Instructions 上，FedHDS 和 FedHDS-Turbo 采用 $\eta = 3 \times 10^{-4}$ ，Random 在使用 DataJuicer-1.3B 时采用 $\eta = 3 \times 10^{-4}$ ，在使用 LLaMA-3B 时采用 $\eta = 3 \times 10^{-5}$ ，Perplexity 在使用 DataJuicer-1.3B 时采用 $\eta = 3 \times 10^{-5}$ ，在使用 LLaMA-3B 时采用 $\eta = 3 \times 10^{-4}$ 。

For Random and Perplexity, we searched for the optimal data ratio within $\{0.2\%, 1.5\%, 2\%, 5\%\}$, where 0.2% and 1.5% are inspired by those automatically obtained by FedHDS and FedHDS-Turbo. Our approaches adopt learning rate $\eta = 3 \times 10^{-5}$ on Dolly-15 K and $\eta = 3 \times 10^{-4}$ on NI. FedHDS and FedHDS-Turbo apply HDBSCAN separately in both intra-client and inter-client selection. During intra-client selection, the minimum cluster of HDBSCAN is set to the default value, i.e., 5 for Natural Instructions and 2 for Dolly-15K, considering the relatively small scale of Dolly-15 K. During inter-

client selection, the minimum cluster of HDBSCAN is uniformly set to 2 .

对于 Random 和 Perplexity, 我们在 $\{0.2\%, 1.5\%, 2\%, 5\%\}$ 范围内搜索了最佳数据比例, 其中 0.2% 和 1.5% 的灵感来自 FedHDS 和 FedHDS-Turbo 自动获取的比例。我们的方法在 Dolly-15K 上采用学习率 $\eta = 3 \times 10^{-5}$, 在 NI 上采用 $\eta = 3 \times 10^{-4}$ 。FedHDS 和 FedHDS-Turbo 在客户端内和客户端间选择中分别应用 HDBSCAN。在客户端内选择期间, 考虑到 Dolly-15K 的规模相对较小, HDBSCAN 的最小簇 (minimum cluster) 在 Natural Instructions 上设置为默认值 5, 在 Dolly-15K 上设置为 2。在客户端间选择期间, HDBSCAN 的最小簇统一设置为 2。

D. 4 Detailed Descriptions on Datasets

D. 4 数据集详细说明

This work adopts the same data preprocessing as reported in (Qin et al., 2024a). Following Zhang et al. (2024); Qin et al. (2024a), we adopt the prompt template from Alpaca (Taori et al., 2023).

本研究采用了与 (Qin et al., 2024a) 中报道相同的预处理方式。遵循 Zhang et al. (2024) 和 Qin et al. (2024a) 的做法, 我们采用了 Alpaca (Taori et al., 2023) 的提示模板。

Natural Instructions includes a large collection of tasks with natural language instructions. We adopt the dataset versioned by v 2.8 and the default split, which includes 756 tasks for training and 119 tasks for testing, each with a distinct task definition. Considering the scale of the dataset,

Natural Instructions 包含大量带有自然语言指令的任务。我们采用了 v2.8 版本的数据集和默认划分方式, 其中包括 756 个训练任务和 119 个测试任务, 每个任务都有独特的任务定义。考虑到数据集的规模,

experiments on it are conducted on a randomly sampled subset, containing 20% of the data instances for each training task and 2% for each test task. After the subsampling, each training task with at least 20 data instances is assigned to a unique client. After the above preprocessing, a federated scenario with 738 clients is formed, where the test tasks remain on the server for the held-out evaluation of the tuned LLMs.

相关实验是在随机采样的子集上进行的, 该子集包含每个训练任务 20% 的数据实例, 以及每个测试任务的 2%。经过子采样后, 每个拥有至少 20 个数据实例的训练任务被分配给一个唯一的客户端。经过上述预处理, 形成了一个拥有 738 个客户端的联邦场景, 其中测试任务保留在服务器上, 用于对微调后的 LLMs 进行留出评估。

Dolly-15K contains 15,015 data samples corresponding to 8 tasks, with the 1,188 data samples from the last task used for testing. The data from the remaining seven tasks is distributed to 200 clients for training, with each task labeled according to its respective task and distributed according to a Dirichlet distribution. We partition data samples to clients via Dirichlet distribution based on the category attribute of each data sample.

Dolly-15K 包含 15,015 个数据样本, 对应 8 个任务, 其中最后一个任务的 1,188 个数据样本用于测试。其余七个任务的数据被分配给 200 个客户端进行训练, 每个任务根据其所属类别进行标记, 并按照狄利克雷分布 (Dirichlet distribution) 进行分发。我们根据每个数据样本的类别属性, 通过狄利克雷分布将数据样本划分给各个客户端。

After the last FL round is finished, Rouge-L scores are evaluated using the tuned LLM after the final FL round. These scores are calculated based on the data samples in evaluation tasks, with the responses as the ground-truth labels.

在最后一轮联邦学习 (FL) 结束后, 使用微调后的 LLM 评估 Rouge-L 分数。这些分数是基于评估任务中的数据样本计算的, 并以其回复作为地面真值 (ground-truth) 标签。

E 补充实验

E. 1 Evaluation of Features from Different Transformer Layers

E. 1 不同 Transformer 层特征的评估

To better demonstrate the statement that the last layer is not universally optimal, and no single layer excels across all metrics in Section 4.2, we conduct evaluations with more LLMs and more metrics including Silhouette Coefficient (Rousseeuw, 1987) and Davies-Bouldin score (Davies and Bouldin, 1979), following the experimental setups aligned to that of Figure 4. The additional experimental results in Figure 9 further demonstrate the above statement.

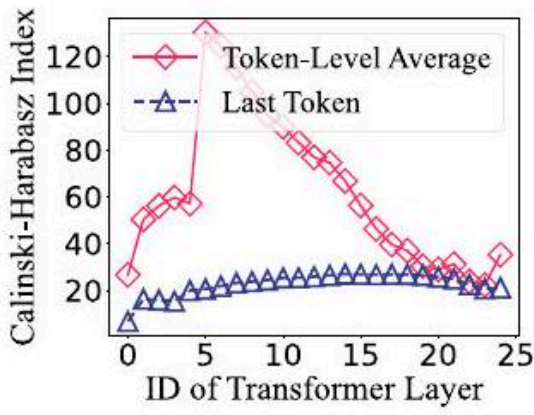
为了更好地证明第 4.2 节中关于“最后一层并非普遍最优，且没有单层能在所有指标上均表现出色”的论述，我们遵循与图 4 一致的实验设置，使用了更多的 LLMs 和更多的指标（包括 Silhouette 系数 (Rousseeuw, 1987) 和 Davies-Bouldin 得分 (Davies and Bouldin, 1979)）进行了评估。图 9 中的额外实验结果进一步证实了上述论述。

E. 2 Virtualization of Data Features

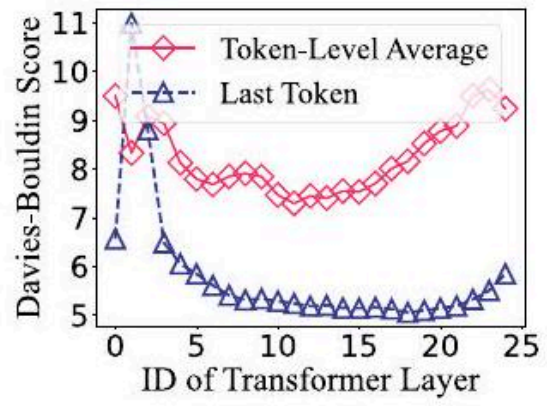
E. 2 数据特征可视化

In Section 4.3, we visualize the features obtained from the last Transformer layer and the fused features with DataJuicer-1.3B on Dolly-15K as an example, as presented in Figure 5. In order to illustrate the differences between fused features and the final layer features in more scenarios, we supplemented the visualizations using different LLMs in various scenarios. As shown in Figure 10, the fused features create more distinct boundaries among clients. Thus, these fused features can be utilized to more effectively distinguish between data samples than existing solutions.

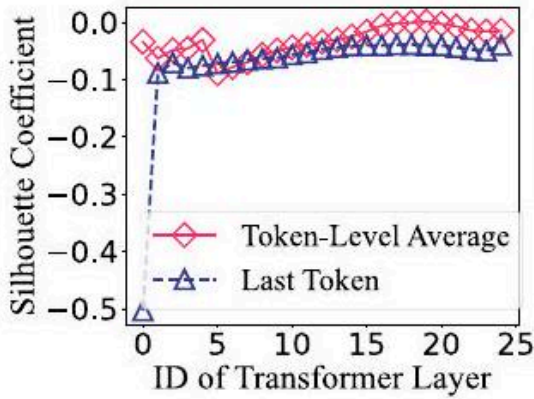
在第 4.3 节中，我们以在 Dolly-15K 数据集上使用 DataJuicer-1.3B 为例，对从最后一个 Transformer 层获取的特征以及融合特征进行了可视化，如图 5 所示。为了说明在更多场景下融合特征与最终层特征之间的差异，我们补充了在不同场景下使用不同 LLMs 的可视化结果。如图 10 所示，融合特征在客户端之间形成了更清晰的边界。因此，与现有解决方案相比，这些融合特征可以更有效地用于区分数据样本。



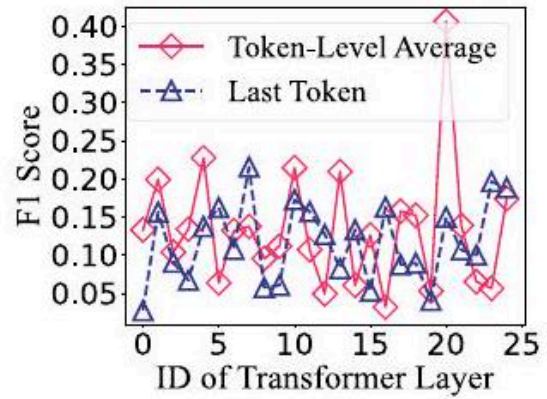
(a) CH Index (1.3B) ↑



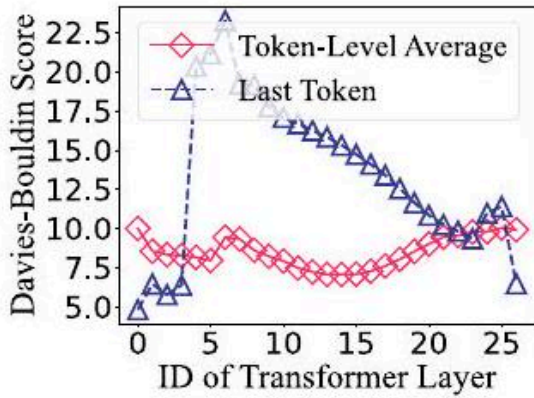
(b) DB Score (1.3B) ↓



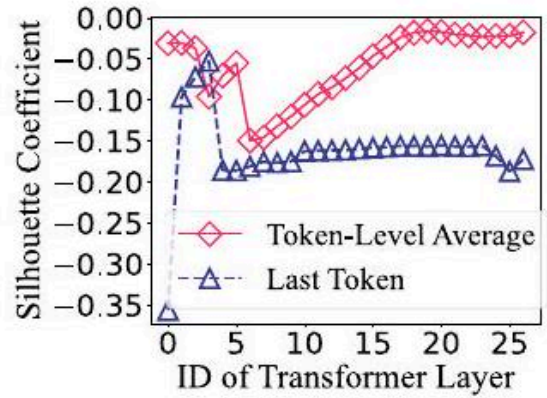
(c) SC Index (1.3B) ↑



(d) F_1 -Score (1.3B) ↑



(e) DB Score (3B) ↓



(f) SC Index (3B) ↑

Figure 8: Figure 9: Evaluations of clustered data groups based on features from different Transformer layers, obtained in a centralized scenario with different LLMs on Dolly-15K.

图 8：图 9：在中心化场景下，基于不同 LLMs 在 Dolly-15K 上获取的来自不同 Transformer 层的特征，对聚类数据组进行的评估。

E. 差分隐私下的性能表现

Despite that 1) the feature dimensions in our approach are significantly lower compared to the original token-level hidden states, which often have thousands of dimensions, and 2) the shared centroids do not correspond to real data samples, FedHDS still provides additional information compared to the standard FL paradigm (McMahan et al., 2017), such as the spatial distribution of client-side centroids. As discussed in Section 4.5, scaling the elements of the shared centroids to the range $[-1, 1]$ and then adding Gaussian noise can enhance the privacy protection of the proposed approaches.

尽管 1) 与通常具有数千维度的原始 token 级隐藏状态相比，我们方法中的特征维度显著降低，且 2) 共享的质心并不对应真实的训练样本，但与标准联邦学习范式 (McMahan et al., 2017) 相比，FedHDS 仍提供了额外的信息，例如客户端质心的空间分布。正如第 4.5 节所讨论的，将共享质心的元素缩放到 $[-1, 1]$ 范围，然后添加高斯噪声，可以增强所提方法的隐私保护能力。

To clarify the impact of adding Gaussian noise, we present example experiments by FedHDS-Turbo on Dolly-15K with DataJuicer-1.3B, adjusting the power of the noise by varying its variance. As presented in Figure 11, when the noise variance is no greater than 0.1, FedHDS-Turbo still outperforms Random. With the noise level increasing further, the performance of FedHDS-Turbo gradually degrades to a level comparable to that of Random as reported in Table 2.

为了阐明添加高斯噪声的影响，我们展示了 FedHDS-Turbo 在 Dolly-15K 数据集上使用 DataJuicer-1.3B 进行的示例实验，通过改变方差来调整噪声强度。如图 11 所示，当噪声方差不大于 0.1 时，FedHDS-Turbo 的表现仍然优于 Random。随着噪声水平的进一步增加，FedHDS-Turbo 的性能逐渐下降，最终达到与表 2 中报告的 Random 相当的水平。

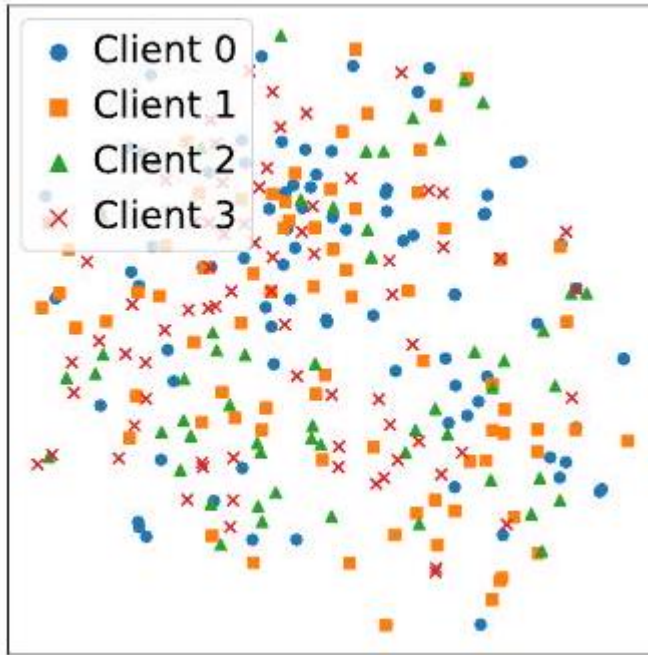


Figure 9: (c) Features by the last Transformer layer with LLaMA3 B in Dolly-15K ($\alpha = 5.0$).

图 9: (c) Dolly-15K 中 LLaMA3 B 最后一个 Transformer 层提取的特征 ($\alpha = 5.0$)。



Figure 10: (e) Features by the last Transformer layer with DataJuicer1.3 B in Dolly-15K ($\alpha = 0.5$).

图 10: (e) 在 Dolly-15K ($\alpha = 0.5$) 上使用 DataJuicer1.3 B 时, 最后一个 Transformer 层的特征。

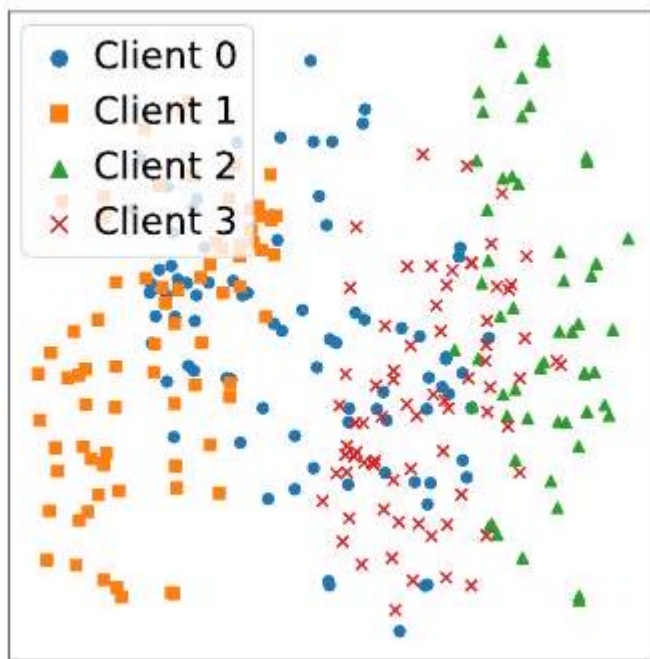


Figure 11: (d) Fused features by FedHDS with LLaMA-3B in Dolly-15K ($\alpha = 5.0$).

图 11: (d) 在 Dolly-15K ($\alpha = 5.0$) 上使用 LLaMA-3B 时, FedHDS 的融合特征。

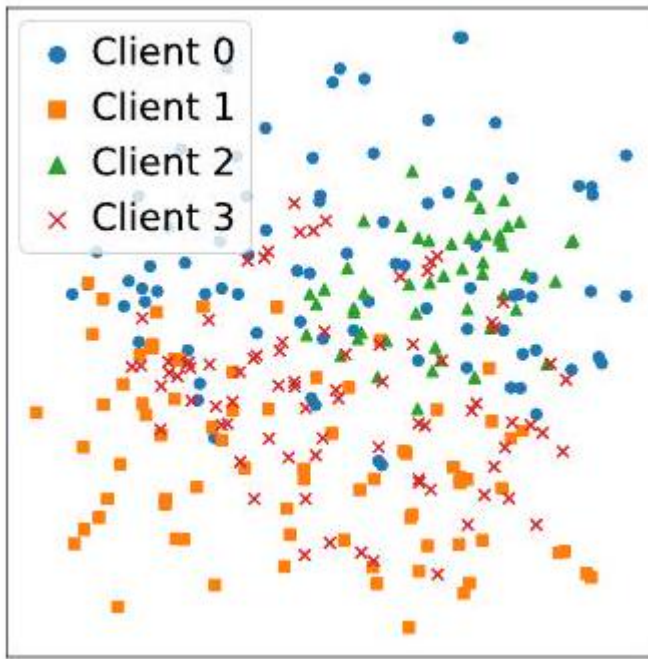


Figure 12: (f) Fused features by FedHDS with DataJuicer-1.3B in Dolly- 15 K($\alpha = 0.5$).

图 12: (f) 在 Dolly- 15 K($\alpha = 0.5$) 上使用 DataJuicer-1.3B 时, FedHDS 的融合特征。

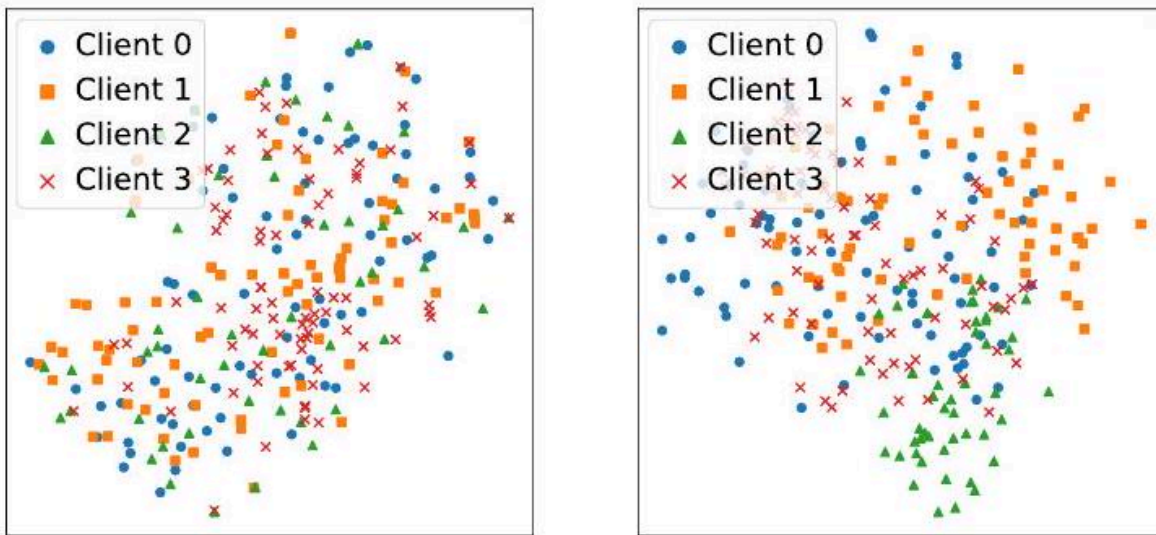


Figure 13: (a) Features by the last Transformer layer with LLaMA3 B in Dolly- 15 K($\alpha = 0.5$).

图 13: (a) 在 Dolly- 15 K($\alpha = 0.5$) 上使用 LLaMA3 B 时, 最后一个 Transformer 层的特征。

(b) Fused features by FedHDS with LLaMA-3B in Dolly- 15 K($\alpha = 0.5$).

(b) 在 Dolly- 15 K($\alpha = 0.5$) 数据集上, 使用 LLaMA-3B 通过 FedHDS 融合后的特征。

Figure 10: Visualization of features from the last Transformer layer and fused features by FedHDS.

图 10: 最后一个 Transformer 层的特征以及通过 FedHDS 融合后的特征的可视化。

E. 4 Breakdown of Time Consumption

E. 4 时间消耗细目

To illustrate the time efficiency of our approach, we provide a detailed breakdown of the time taken by each step, recorded on Natural Instructions (Table 6) and Dolly-15K (Table 7), respectively. From these results, we have the following observations:

为了说明我们方法的时间效率，我们提供了每个步骤所用时间的详细细目，分别记录在 Natural Instructions（表 6）和 Dolly-15K（表 7）上。从这些结果中，我们得出以下观察结果：

For FedHDS, the time consumption is usually dominated by feature extraction, due to the relatively high time cost of performing inference with an LLM on all local data samples.

对于 FedHDS 而言，时间消耗通常由特征提取主导，这是因为在所有本地数据样本上使用 LLM 进行推理的时间成本相对较高。

For FedHDS-Turbo, due to its faster feature extraction than FedHDS, its time consumption is mainly caused by training, feature extraction, and feature fusion.

对于 FedHDS-Turbo，由于其特征提取速度快于 FedHDS，其时间消耗主要由训练、特征提取和特征融合产生。

Table 6: Table 6: Breakdown of time consumption for all steps involving with FedHDS and FedHDS-Turbo on Natural Instructions. By default, this table shows the time spent per FL round for each client, with the total time across all clients and rounds in parentheses.

表 6：在 Natural Instructions 数据集上，FedHDS 和 FedHDS-Turbo 涉及的所有步骤的时间消耗细分。默认情况下，本表显示每个客户端在每个联邦学习（FL）轮次中所花费的时间，括号内为所有客户端和所有轮次的总时间。

	DataJuicer-1.3B on NI 1.3B	在 NI 数据集上的 DataJuicer-1.3B	LLaMA-3B on NI	LLaMA-3B 在 NI 数据集上
FedHDS	All Steps: 27.5S (11H17M) Training: 0.2 S (3M55S) Feature Extraction: 24.8 S(10H11M) Feature Fusion: 2.5 S(1H1M) Intra-Client Clustering: 6.7×10^{-3} S (9.9S) Inter-Client Clustering: 4.3×10^{-3} S(0.2 S)	总耗时: 27.5S (11H17M) 训练: 0.2 S (3M55S) 特征提取: 24.8 S(10H11M) 特征融合: 2.5 S(1H1M) 客户端内聚类: 6.7×10^{-3} S (9.9S) 客户端间聚类: 4.3×10^{-3} S(0.2 S)	All Steps: 40.7S (16H42M) Training: 6.8 S(2H47M) Feature Extraction: 30.9S (12 H 42 M) Feature Fusion: 3.0S (1 H 13 M) Intra-Client Clustering: 6.6×10^{-3} S (9.8S) Inter-Client Clustering: 3.6×10^{-3} S(0.1 S)	总耗时: 40.7S (16H42M) 训练: 6.8 S(2H47M) 特征提取: 30.9S (12 H 42 M) 特征融合: 3.0S (1 H 13 M) 客户端内聚类: 6.6×10^{-3} S (9.8S) 客户端间聚类: 3.6×10^{-3} S(0.1 S)
FedHDS-Turbo	All Steps: 10.0S (4H7M) Training: 1.4S (34M12S) Feature Extraction: 6.6S (2H43M) Feature Fusion: 2.0S (49M26S) Intra-Client Clustering: 6.7×10^{-3} S (9.9S) Inter-Client Clustering: 4.3×10^{-3} S(0.2 S)	所有步骤: 10.0 秒 (4 小时 7 分) 训练: 1.4 秒 (34 分 12 秒) 特征提取: 6.6 秒 (2 小时 43 分) 特征融合: 2.0 秒 (49 分 26 秒) 客户端内聚类: 6.7×10^{-3} S (9.9 秒) 客户端间聚类: 4.3×10^{-3} S(0.2 S)	All Steps: 20.1S (8 H 15 M) Training: 11.4 S(4H40M) Feature Extraction: 6.7S (2H46M) Feature Fusion: 2.0S (48M20S) Intra-Client Clustering: 6.6×10^{-3} S (9.7S) Inter-Client Clustering: 4.6×10^{-3} S(0.2 S)	所有步骤: 20.1 秒 (8 小时 15 分) 训练: 11.4 S(4H40M) 特征提取: 6.7 秒 (2 小时 46 分) 特征融合: 2.0 秒 (48 分 20 秒) 客户端内聚类: 6.6×10^{-3} S (9.7 秒) 客户端间聚类: 4.6×10^{-3} S(0.2 S)

Table 7: Table 7: Breakdown of time consumption for all steps involving with FedHDS and FedHDS-Turbo on Dolly-15K. By default, this table shows the time spent per FL round for each client, with the total time across all clients and rounds in

parentheses.

表 7：在 Dolly-15K 数据集上涉及 FedHDS 和 FedHDS-Turbo 的所有步骤的时间消耗明细。默认情况下，本表显示每个客户端在每轮联邦学习（FL）中花费的时间，括号内为所有客户端和所有轮次的总时间。

	DataJuicer-1.3B on Dolly-15K DataJuicer-1.3B 在 Dolly-15K 上	LLaMA-3B on Dolly-15K LLaMA-3B 在 Dolly-15K 上
FedHDS	All Steps: 5.2S (34M52S) 所有步骤: 5.2S (34M52S)	All Steps: 14.6 S(2H25M) 总步数: 14.6 S(2H25M)
	Training: 2.0S (13M23S) 训练: 2.0S (13M23S)	Training: 10.2 S(1H41M) 训练: 10.2 S(1H41M)
	Feature Extraction: 3.0 S (19M42S) 特征提取: 3.0 S (19M42S)	Feature Extraction: 4.1S(40M37S) 特征提取: 4.1S (40M37S)
	Feature Fusion: 0.3 S (1M46S) 特征融合: 0.3 S (1M46S)	Feature Fusion: 0.3S (3M21S) 特征融合: 0.3S (3M21S)
	Intra-Client Clustering: 1.6×10^{-3} S(0.6 S) 客户端内聚类: 1.6×10^{-3} S(0.6 S)	Intra-Client Clustering: 1.7×10^{-3} S(1.0 S) 客户端内聚类: 1.7×10^{-3} S(1.0 S)
	Inter-Client Clustering: 8.5×10^{-4} S (3.4×10^{-2} S) 客户端间聚类: 8.5×10^{-4} S (3.4×10^{-2} S)	Inter-Client Clustering: 1.0×10^{-3} S (6.2×10^{-2} S) 客户端间聚类: 1.0×10^{-3} S (6.2×10^{-2} S)
FedHDS-Turbo	All Steps: 4.5S(29M42S) 所有步骤: 4.5 秒 (29 分 42 秒)	All Steps: 17.2 S (2 H 52 M) 所有步骤: 17.2 秒 (2 小时 52 分)
	Training: 3.6 S (23M45S) 训练: 3.6 秒 (23 分 45 秒)	Training: 16.3 S(2H43M) 训练: 16.3 S(2H43M)
	Feature Extraction: 0.7 S (4M40S) 特征提取: 0.7 秒 (4 分 40 秒)	Feature Extraction: 0.7 S (6M54S) 特征提取: 0.7 秒 (6 分 54 秒)
	Feature Fusion: 0.2 S (1M15S) 特征融合: 0.2 秒 (1 分 15 秒)	Feature Fusion: 0.2S (1M49S) 特征融合: 0.2 秒 (1 分 49 秒)
	Intra-Client Clustering: 1.6×10^{-3} S(0.6 S) 客户端内聚类: 1.6×10^{-3} S(0.6 S)	Intra-Client Clustering: 1.8×10^{-3} S(1.1 S) 客户端内聚类: 1.8×10^{-3} S(1.1 S)
	Inter-Client Clustering: 1.0×10^{-3} S (4.0×10^{-2} S) 客户端间聚类: 1.0×10^{-3} S (4.0×10^{-2} S)	Inter-Client Clustering: 1.7×10^{-3} S(0.1 S) 客户端间聚类: 1.7×10^{-3} S(0.1 S)

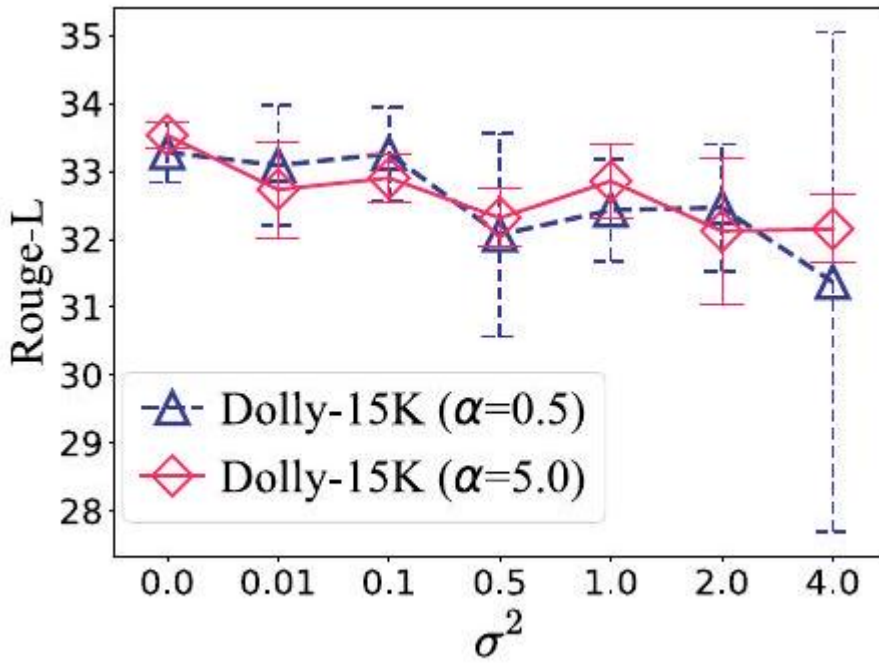


Figure 14: Figure 11: Effects of adding DP noise to FedHDS-Turbo (DataJuicer-1.3B).

图 14: 图 11: 在 FedHDS-Turbo (DataJuicer-1.3B) 中添加 DP 噪声的效果。

E. 5 Performance in Various FL Scenarios

E. 5 不同联邦学习 (FL) 场景下的性能

The ratio of active clients in each round of FL affects the number of centroids sent to the server, potentially affecting the effectiveness of inter-client selection. We test approaches with coresets in varying proportions of active clients in each round. As shown in Figure 12, FedHDS-Turbo consistently outperforms Random and Perplexity across varying active client ratios. When the active client ratio is low (1%), all approaches perform unsatisfacto-

每一轮联邦学习中活跃客户端的比例会影响发送到服务器的质心数量，从而可能影响客户端间选择的效果。我们测试了在不同活跃客户端比例下，使用核心集（coresets）的方法。如图 12 所示，在不同的活跃客户端比例下，FedHDS-Turbo 的表现始终优于 Random 和 Perplexity。当活跃客户端比例较低（1%）时，所有方法的表现均不理想。

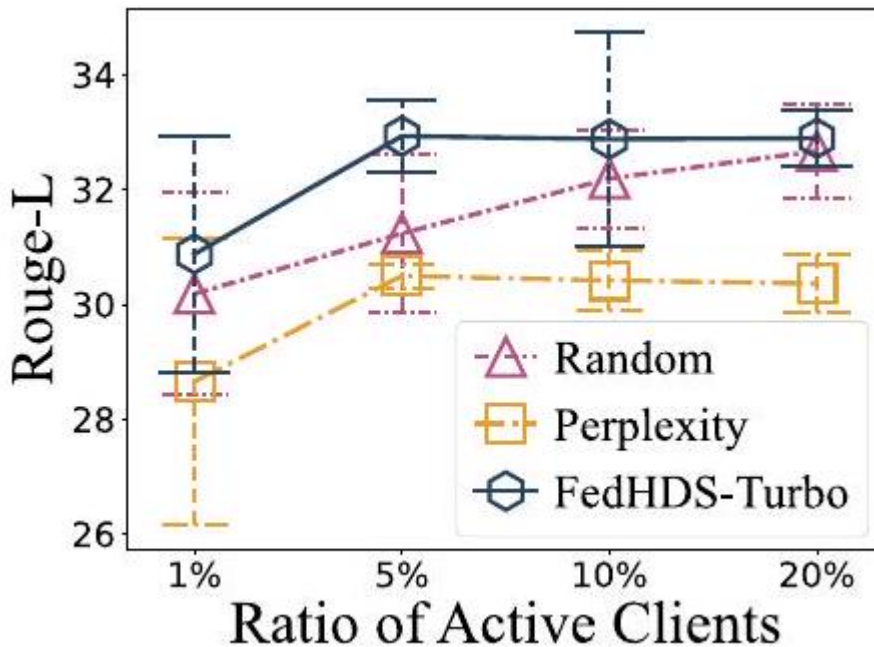


Figure 15: Figure 12: Rouge-L with different active ratio on NI with LLaMA-3B.

图 15: 图 12: 在 NI 数据集上使用 LLaMA-3B 时, 不同活跃比例下的 Rouge-L 指标。

rily, where the effectiveness of intra-client selection in FedHDS-Turbo may suffer from insufficient centroids. With an increasing active client ratio, Random demonstrates a robust growth trend, although it still lags behind FedHDS-Turbo. This improvement is likely due to when there are more active clients in each round, the randomly sampled data instances follow a distribution more aligned to the global data distribution. Considering that clients typically participate in FL with a relatively

在 FedHDS-Turbo 中, 由于质心数量不足, 客户端内选择的有效性可能会受到影响。随着活跃客户端比例的增加, Random 表现出强劲的增长趋势, 尽管它仍然落后于 FedHDS-Turbo。这种提升可能是因为当每轮有更多活跃客户端时, 随机采样的数实例所遵循的分布会更趋近于全局数据分布。考虑到客户端通常以相对较低的参与度加入联邦学习 (FL)

low active ratio in each round in cross-device FL scenarios (McMahan et al., 2017; Qin et al., 2024a; Xu et al., 2024), FedHDS-Turbo is more suitable than FedHDS for cross-device settings in terms of both accuracy and efficiency.

在跨设备联邦学习场景中, 由于每一轮的激活率较低 (McMahan et al., 2017; Qin et al., 2024a; Xu et al., 2024), FedHDS-Turbo 在准确性和效率方面都比 FedHDS 更适合跨设备环境。

F Detailed Calculation of Communication Overhead

F 通信开销的详细计算

Apart from the transmission of model parameters as other federated instruction tuning methods, FedHDS additionally transmits data features of cluster centers and the indices of the selected clusters for data selection. Assuming for a client, there are ρ clusters after intra-client selection. The client sends the fused features of the ρ data samples closest to these cluster centers, as:

除了像其他联邦指令微调方法那样传输模型参数外, FedHDS 还额外传输聚类中心的数据特征以及用于数据选择的所选聚类索引。假设对于一个客户端, 在客户端内选择后存在 ρ 个聚类。客户端发送与这些聚类中心最接近的 ρ 个数据样本的融合特征, 如下所示:

$$[[e_{1,1}, e_{1,2}], [e_{2,1}, e_{2,2}], \dots, [e_{\rho,1}, e_{\rho,2}]]$$

where e denotes a floating number, and $[]$ denotes an array. Then, after the inter-client selection, the server returns indices of selected clusters to corresponding clients, which are just a few integers:

其中 e 表示浮点数, $[]$ 表示数组。随后, 在客户端间筛选之后, 服务器将所选簇的索引返回给相应的客户端, 这些索引仅为几个整数:

$$[\text{ClusterID}_1, \text{ClusterID}_2, \dots].$$

Therefore, compared to the widely recognized baseline method FedIT, the additional communication overhead of our approach is negligible.

因此, 与广受认可的基准方法 FedIT 相比, 我们方法的额外通信开销微乎其微。

*Corresponding authors. *通讯作者。

¹ CPU+GPU training is implemented with Deepspeed (Rasley et al., 2020) and a batch size of 1 (loaded in 16-bit).

¹ CPU+GPU 训练采用 Deepspeed (Rasley et al., 2020) 实现, 批量大小 (batch size) 为 1 (以 16 位精度加载)。

² The most popular desktop and laptop GPUs are reported by Steam Hardware Survey (Dec 2024).

² Steam 硬件调查 (2024 年 12 月) 报告了最受欢迎的台式机和笔记本电脑 GPU。

¹ Our codes are available at <https://github.com/zhenqincn/FedHDS>.

¹ 我们的代码可在 <https://github.com/zhenqincn/FedHDS> 获取。